Executable Command Languages

Trees, Scopes, Semantics, and Interpreters: A Computer Science and Mathematical Study of Cobra

Pedagogical reconstruction from the Cobra Software Architecture Garden study

2026-08-16

Contents

Preface

A mature command-line program is not merely a collection of callbacks behind a parser. It is a small programming language. It has a vocabulary of commands and flags, a hierarchical grammar, scope rules, static constraints, operational semantics, diagnostic behavior, interactive tooling, and reference documentation. Once a CLI grows beyond a few verbs, the same questions appear that arise in compilers, interpreters, configuration languages, workflow engines, and protocol servers:

- What is the authoritative representation of the language?
- How are names resolved in a hierarchy?
- Which values are local, inherited, or shadowed?
- At what point is an invocation admitted to perform effects?
- How can validation, completion, help, and documentation agree without duplicating the schema?
- How can the whole system be tested without launching a subprocess for every case?

This book studies those questions through the architecture of the Go library `spf13/cobra`. The aim is not to teach Cobra's surface API as a cookbook. The aim is to extract the computer-science structures beneath that API and make them reusable. Cobra is especially instructive because one mutable command graph is interpreted by routing, flag processing, validation, lifecycle hooks, shell completion, help, tests, and documentation generation. That architecture exposes both strong patterns and important non-guarantees.

The source study is pinned to Cobra commit `adbc8813901bba65827259daa8e22ff94ec1f30e`. The architectural observations were first organized in the Cobra Software Architecture Garden study and its eight focused pattern notes. This textbook rewrites that material from first principles, adds a continuous running example, introduces formal definitions only after motivating them, and applies each definition in worked examples before asking the reader to use it in exercises.

Intended audience

The primary audience is an advanced undergraduate or graduate student in computer science, or a software engineer who wants a formal vocabulary for interface frameworks. Familiarity with basic data structures, functions, Boolean logic, and Go-like pseudocode is helpful. No knowledge of Cobra is assumed.

How to read the book

The four chapters form a dependency chain.

1. **Command graphs as executable schemas** models a CLI as a rooted, labeled tree and introduces the idea of one model interpreted in several ways.
2. **Scopes, inheritance, shadowing, and late binding** adds environments and phase-sensitive defaults to that tree.

3. **Operational semantics of command execution** turns an invocation into a transition system with explicit admission predicates, hook order, and error boundaries.
4. **Multiple interpreters over one model** shows how declarative constraints, completion protocols, tests, and generated documentation reuse the same semantic structure.

Each chapter follows the same pedagogical sequence: motivation, definitions, worked examples, counterexamples or failure modes, implementation correspondence, and exercises. Selected hints and solutions appear after the four chapters.

The running example: `atlas`

We will use one fictional command-line application throughout the book. `atlas` manages projects and a local cache:

```
atlas [--config FILE] [--verbose]
  project
    list [--output table|json]
    add NAME [--region REGION] [--template FILE]
  cache
    clear [--all | --key KEY]
  completion SHELL
```

The same example will be viewed as a tree, a scoped environment, a state machine, a Boolean constraint system, a completion protocol, and a documentation source. Reusing one example matters: the reader can see that these are not separate tricks but different interpretations of one structure.

> **Notation.** We write finite sequences with angle brackets, such as $\langle \texttt{project}, \texttt{add} \rangle$. A partial function is written $f : A \rightharpoonup B$; it may be undefined for some inputs. For a node v, $\mathrm{parent}(v)$ is its parent and $\mathrm{path}(v)$ is the sequence of command names from the root to v.

Chapter 1

Command Graphs as Executable Schemas

1.1 Learning objectives

By the end of this chapter, you should be able to:

- model a hierarchical CLI as a rooted, labeled tree with metadata;
- define command paths, aliases, and resolution as partial functions;
- explain why dispatch, help, completion, validation, and documentation are separate interpreters over one model;
- use tree folds to describe recursive tooling;
- identify ambiguity, phase, mutability, and authorization counterexamples.

1.2 Why a CLI should be treated as a language

Imagine implementing `atlas` with five separate data structures:

1. a parser table that knows which subcommands exist;
2. a help catalog that lists commands and descriptions;
3. a completion file that repeats command and flag names;
4. a validation table that describes valid arguments;
5. a documentation hierarchy maintained by hand.

At first these structures agree. Six months later, `project archive` is added to the parser but not to completion. A flag is renamed in code but remains in the manual. A deprecated command disappears from help but is still suggested by the shell. Each copy is individually simple; the system is collectively inconsistent.

The central architectural response is to model the interface once and interpret it many times. Cobra's `Command` graph is such a model. It contains topology, names, aliases, descriptions, flags, validators, lifecycle functions, completion providers, context, and I/O overrides. Routing, help, completion, validation, and documentation ask that graph different questions rather than maintaining sibling schemas.

1.3 From command line to formal language

A command line consists of tokens. Some tokens name commands, some are flags, some are values, and some are positional arguments. Before we discuss flags or execution, we isolate the

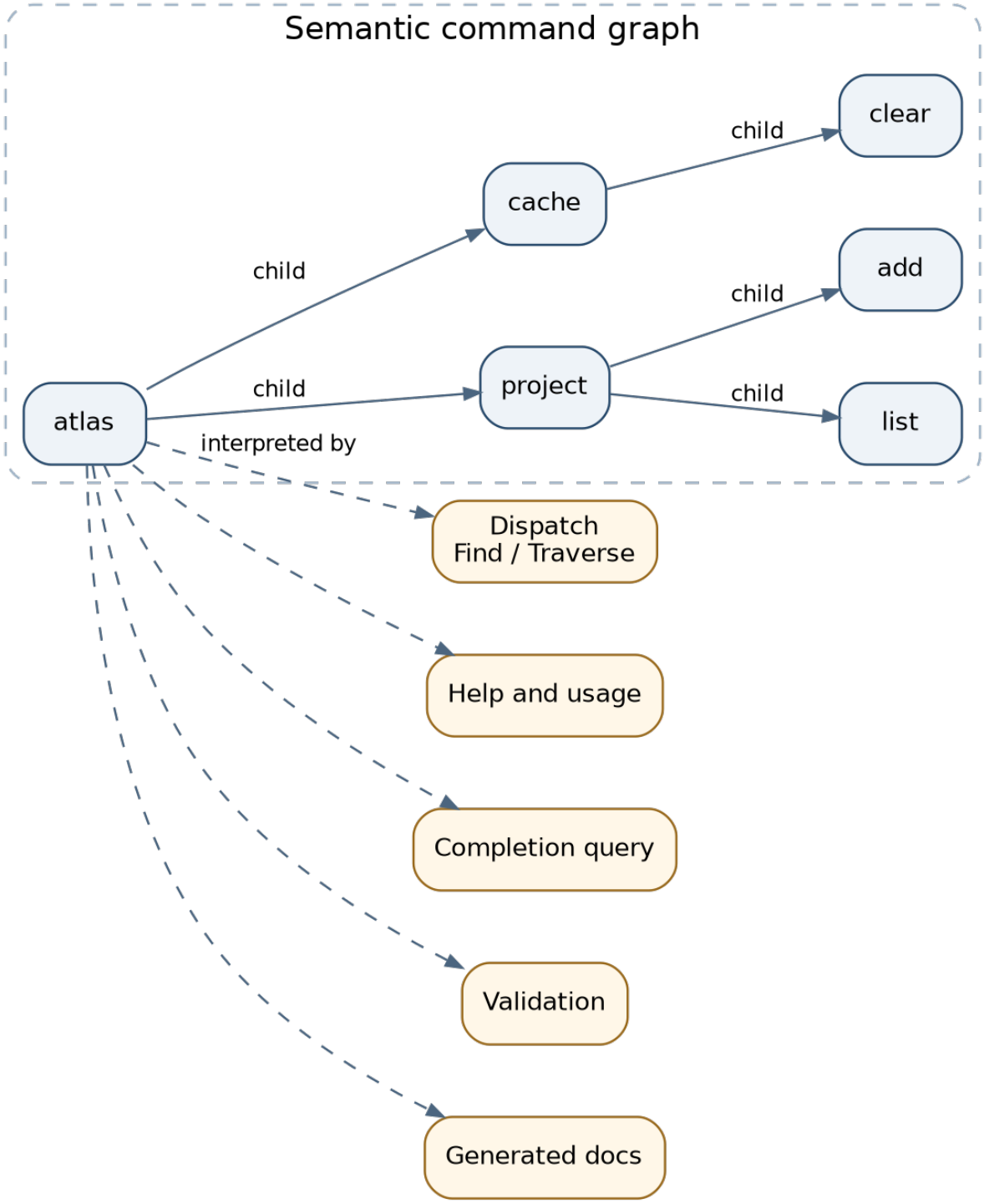

Figure 1.1: One semantic command graph interpreted by dispatch, help, validation, completion, and documentation.

part that chooses a command.

Definition 1.1 (Token alphabet and command vocabulary). Let Σ be the set of lexical tokens that may occur on a command line. Let $\Sigma_C \subseteq \Sigma$ be the finite subset used as command-name tokens, such as

$$\Sigma_C = \{\texttt{project}, \texttt{list}, \texttt{add}, \texttt{cache}, \texttt{clear}\}.$$

A **command word** is a finite sequence in Σ_C^*. For example,

$$w = \langle \texttt{project}, \texttt{add} \rangle$$

is a command word selecting the `add` command beneath `project`.

A CLI is not usually a flat language: `add` may be valid under `project` but not under `cache`. The structure is therefore hierarchical.

Definition 1.2 (Semantic command graph). A semantic command graph is a tuple

$$T = (V, E, r, \lambda, \mu),$$

where:

- V is a finite set of command nodes;
- $E \subseteq V \times V$ is a parent-to-child relation;
- $r \in V$ is the root command;
- $\lambda : V \to \Sigma_C$ assigns a primary name to each node;
- $\mu : V \to M$ assigns command metadata, such as aliases, descriptions, flags, validators, and handlers.

For ordinary Cobra composition, (V, E, r) is intended to be a rooted tree: every node except r has exactly one parent, and every node is reachable from r.

> **Fundamentals - Trees, graphs, and abstract syntax trees.** A rooted tree is a directed acyclic graph with one distinguished root and exactly one path from the root to each node. An abstract syntax tree represents the grammatical structure of one program. A command tree differs slightly: it represents the grammar of *all* valid command paths, not one invocation. An invocation selects a path through that grammar.

The tree property gives each node a unique structural identity.

Definition 1.3 (Command path). For a node v, the command path is the unique root-to-node sequence

$$\text{path}(v) = \langle \lambda(v_1), \lambda(v_2), \ldots, \lambda(v_k) \rangle,$$

where v_1 is a child of the root and $v_k = v$. The root's executable name may be included when displaying the full path, but it is not required for internal descent.

For `atlas`, the command paths include

```
<project>
<project, list>
<project, add>
<cache>
<cache, clear>
```

The path is not merely a display string. It determines scope, inheritance, help navigation, completion candidates, and generated documentation links.

1.4 Dispatch as a partial function

The next problem is name resolution: given a command word, which node does it denote?

Definition 1.4 (Exact child lookup). For a node v, define

$$\mathrm{child}_T(v, s) : V \times \Sigma_C \rightharpoonup V$$

so that $\mathrm{child}_T(v, s) = u$ exactly when $(v, u) \in E$ and $\lambda(u) = s$. The function is partial because not every token names a child at every node.

Aliases extend the lookup relation. Let $A(u) \subseteq \Sigma_C$ be the alias set of u.

Definition 1.5 (Name match). A token s matches a command node u when

$$\mathrm{match}(u, s) \iff s = \lambda(u) \vee s \in A(u).$$

Cobra can also enable prefix and case-insensitive matching. Those are not harmless presentation options: they change the matching relation and therefore the language accepted by the dispatcher.

Definition 1.6 (Command resolution). The resolver

$$\mathrm{resolve}_T : \Sigma^* \rightharpoonup V \times \Sigma^*$$

consumes command-name tokens while descending from the root and returns the selected command node together with the remaining tokens. Flags and their values must be skipped or parsed according to the chosen traversal policy.

In pseudocode:

```
resolve(node, tokens):
    remaining := tokens
    current := node

    loop:
        visible := remove_or_skip_flags(current, remaining)
        if visible has no non-flag token:
            return (current, remaining)

        name := first non-flag token in visible
        next := unique child of current matching name
        if next does not exist:
            return (current, remaining) or an unknown-command error

        remaining := remove the consumed command token,
                     without removing a flag value that happens to equal it
        current := next
```

The subtle phrase “without removing a flag value that happens to equal it” matters. Token sequences are not typeless bags. A token equal to `project` may be a command name in one position and the value of `--name` in another.

> **Fundamentals - Partial functions and errors.** A resolver is naturally partial because some token sequences do not denote commands. An implementation can represent undefinedness as an error, an option type, or a pair containing the deepest resolved node and an error. Returning the deepest node is operationally useful because error messages can mention the relevant subcommand rather than only the root.

1.5 Worked example 1: resolving an invocation

Consider

`atlas --config dev.yaml project add mars --region us-east`

Suppose `--config` is a persistent flag declared on the root. The command-name projection is

$$\langle \texttt{project}, \texttt{add} \rangle.$$

Resolution proceeds as follows:

1. Begin at root node `atlas`.
2. Recognize `--config dev.yaml` as a flag/value pair, not command tokens.
3. Match `project` to the `project` child.
4. Remove the *command occurrence* of `project` from the remaining token sequence.
5. At the `project` node, match `add` to the `add` child.
6. Stop because the remaining non-flag token `mars` is a positional argument, not a child command.
7. Return the `add` node and the arguments/flags that must be parsed and validated there.

A useful resolution trace table is:

Step	Current node	Remaining tokens	Decision
0	`atlas`	`--config dev.yaml project add mars --region us-east`	skip root flag and value
1	`atlas`	`project add mars --region us-east`	descend to `project`
2	`project`	`add mars --region us-east`	descend to `add`
3	`add`	`mars --region us-east`	no matching child; selected command found

This example illustrates why dispatch and flag semantics cannot be entirely separated. The parser must know enough about the current scope to distinguish command tokens from flag values.

1.6 The API correspondence

The mathematical tree corresponds closely to a small subset of Cobra's public API:

```go
type Command struct {
    Use     string
    Aliases []string
    Args    PositionalArgs
    RunE    func(cmd *Command, args []string) error
    // descriptions, flags, hooks, context, streams, children, parent, ...
}

func (c *Command) AddCommand(cmds ...*Command)
func (c *Command) Parent() *Command
func (c *Command) Root() *Command
func (c *Command) CommandPath() string
```

```
func (c *Command) Find(args []string) (*Command, []string, error)
func (c *Command) Traverse(args []string) (*Command, []string, error)
```

The exact `Command` struct is broad and mutable. The transferable structure is not "put everything in one giant object." It is "keep one semantic authority for facts that several interpreters must agree on." A different framework might use an immutable declarative tree plus a separate executor state.

1.7 One model, several interpreters

A tree becomes architecturally valuable when several consumers interpret it.

Definition 1.7 (Interpreter over a command graph). An interpreter is a function

$$I : T \times X \to Y$$

that reads the semantic command graph, possibly with an auxiliary input X, and produces a result Y without maintaining an independent copy of the command schema.

Examples include:

- dispatch: $I_D(T, \text{argv}) = (v, \text{remaining})$;
- help: $I_H(T, v) =$ formatted help text;
- completion: $I_C(T, \text{partial argv}) =$ candidate set and directive;
- documentation: $I_M(T) =$ a family of Markdown pages;
- validation: $I_V(T, v, \text{assignment}) =$ success or error.

These interpreters do not produce the same output. They share only the facts that must agree: command identity, hierarchy, availability, flags, descriptions, and constraints.

Definition 1.8 (Semantic authority). A model T is a semantic authority for a fact p when every subsystem that claims to know p derives it from T or from a documented projection of T, rather than maintaining an independently editable copy.

The key law is a non-drift law:

$$\forall I_i, I_j,\ p \text{ is shared by } I_i, I_j \implies I_i \text{ and } I_j \text{ query the same authoritative representation of } p.$$

This is not a proof that all interpreters are correct. Two buggy functions can read the same model. The law removes one important source of disagreement: duplicated schema ownership.

1.8 Worked example 2: adding a command once

Suppose we add `atlas project archive NAME`. In a graph-centered design, the application adds one node:

```
archive := &cobra.Command{
    Use:   "archive NAME",
    Short: "Archive a project",
    Args:  cobra.ExactArgs(1),
    RunE:  archiveProject,
}
project.AddCommand(archive)
```

From that one structural change:

- dispatch can resolve `project archive`;
- `project --help` can list `archive`;
- command-name completion can suggest `archive`;
- documentation generation can emit an archive page and link it from the project page;
- availability rules can hide or deprecate it consistently.

The handler and metadata may still be wrong, but there is no second command catalog to update.

1.9 Tree folds and recursive interpreters

Many interpreters over a command tree are structurally recursive. For example, documentation generation can be described as a fold.

Definition 1.9 (Tree fold). Let F combine one node's metadata with the already-interpreted results of its children. The fold fold_F is defined recursively by

$$\text{fold}_F(v) = F\left(\mu(v), [\text{fold}_F(c) \mid c \in \text{children}(v)]\right).$$

A help renderer may use only the immediate children. A documentation generator may recursively produce one artifact per node. A validator may not need to visit all children at all. "One model, many interpreters" does not imply "one traversal algorithm."

> **Side topic - Catamorphisms.** In functional programming and category theory, a fold that replaces each constructor of a recursive data type with an algebra is called a catamorphism. You do not need category theory to use the idea. The practical lesson is that a recursive model invites reusable, structurally complete traversals instead of ad hoc searches.

1.10 Mutability and model phases

Cobra's graph is mutable. Commands can be added and removed; derived flag views are cached; default help and completion nodes can be synthesized; completion may temporarily alter flags to guide suggestions. Therefore the phrase "the command graph" is incomplete unless we also name the phase.

Definition 1.10 (Model phase). A model phase is a set of construction or interpretation steps after which a particular view of the graph is considered resolved for a purpose. Typical phases are:

1. application declaration;
2. plugin registration;
3. framework-default synthesis;
4. execution-time parsing and validation;
5. completion-time advisory mutation;
6. documentation-time traversal.

Two interpreters can read the same object and still disagree if they read different phases. Model sharing removes schema duplication, but phase discipline is still required.

A useful ownership rule is:

> Build or mutate the graph under a clear owner. Execute or traverse it under a stable topology unless dynamic mutation is explicitly part of the contract.

1.11 Counterexample 1: duplicated schema ownership

Consider an application that routes commands from a Go map but generates completion from a YAML file:

```
Go router:  project add, project list, project archive
YAML file:  project add, project list
```

The YAML can be perfectly valid and still be semantically stale. A test that parses the YAML cannot detect the missing command unless it compares against the router. The system has created a cross-representation consistency obligation.

The graph-centered alternative does not eliminate tests. It changes what must be tested: instead of checking synchronization between two manually edited schemas, tests check that each interpreter correctly traverses one schema.

1.12 Counterexample 2: ambiguous prefix matching

Suppose the `project` node has children `archive` and `artifacts`. If prefix matching accepts any unique prefix, then `ar` is ambiguous. A resolver that silently chooses the first child converts ordering into semantics.

Formally, let

$$M(v, s) = \{u \in \text{children}(v) \mid s \text{ is a prefix of } \lambda(u)\}.$$

Prefix matching is safe only when $|M(v, s)| = 1$. If $|M(v, s)| > 1$, the resolver must reject or request more input. This example shows how a convenience feature changes the accepted language and introduces an ambiguity proof obligation.

1.13 Counterexample 3: graph identity is not authorization

A command graph answers "what command does this token sequence denote?" It does not answer "may this principal perform the command?" Routing is a syntactic or semantic lookup; authorization is a policy decision over a principal, resource, action, and context.

The distinction can be written as:

$$\text{resolve}_T(\text{argv}) = v \not\Rightarrow \text{authorized}(\text{principal}, v, \text{context}).$$

Treating discoverability as authority is a serious category error. Later chapters will repeatedly separate advisory or structural interpreters from effect authority.

1.14 Design checklist

Before adopting a semantic command graph, ask:

- Which facts must dispatch, help, completion, validation, and docs share?
- Is the structure truly a tree, or do commands have multiple parents?
- Which mutations are allowed, and in which phase?
- Which derived caches require invalidation after mutation?
- Do aliases or prefix matching preserve unambiguous resolution?
- Which concerns should remain outside the graph, such as authorization or domain services?

1.15 Chapter summary

A substantial CLI is a hierarchical language. A rooted, labeled command tree gives command paths unique structural meaning. Dispatch is a partial resolution function over token sequences. Help, completion, validation, and documentation are separate interpreters over the same semantic authority. This architecture reduces schema drift, but it does not imply immutability, authorization, or concurrency safety. The phase at which an interpreter observes a mutable graph is part of the contract.

> **Source correspondence.** The concrete evidence for this chapter is concentrated in Cobra's `command.go`, `completions.go`, `doc/md_docs.go`, `command_test.go`, and `completions_test.go`. The final source map gives stable commit links.

1.16 Exercises

1.16.1 Concepts

1. **Model the running example.** Write the tuple (V, E, r, λ) for the `atlas` command graph. List every root-to-node command path.
2. **Tree or DAG?** Suppose the same `status` command object is attached beneath both `project` and `cache`. Which tree property fails? Give two possible redesigns that preserve a unique path identity.
3. **Aliases.** Add alias `ls` to `project list`. Define the modified match relation and explain whether the canonical command path should contain `ls` or `list`.
4. **Availability.** Define a predicate visible(v) using `Hidden`, `Deprecated`, and "runnable or has visible descendants." Which interpreters should use it, and which might need a different predicate?

1.16.2 Worked reasoning

5. **Resolution trace.** Trace the resolver for

   ```
   atlas project --region us-west add mars
   ```

   under two policies: (a) parent-local flags cannot appear before a child command; (b) traversal parses parent flags while descending. State where the traces diverge.

6. **Ambiguity proof.** For child names `archive`, `artifacts`, and `audit`, characterize all prefixes that uniquely identify one child. Write an algorithm and give its worst-case complexity in the total length of child names.

7. **Anti-drift property.** State a testable property ensuring every visible routable command appears in generated documentation. What inputs and outputs would the test compare?

1.16.3 Design and programming

8. **Interpreter interface.** Design a language-neutral interface for an interpreter over a command graph. Your interface should support a read-only traversal without exposing mutation.
9. **Phase discipline.** Design an immutable alternative to Cobra's late graph mutation. Where would help, version, and completion defaults be resolved? What would be gained and lost?
10. **Counterexample construction.** Construct a duplicated-schema system in which parser and documentation both pass their own unit tests but disagree at runtime. Then write an integration test that exposes the disagreement.

Chapter 2

Scopes, Inheritance, Shadowing, and Late Binding

2.1 Learning objectives

By the end of this chapter, you should be able to:

- represent node-local policy as partial maps;
- calculate effective policy with left-biased override;
- distinguish local, inherited, effective, and provenance views;
- prove the nearest-declaration property;
- state preservation, idempotence, and phase laws for late defaults;
- explain why inheritance does not imply authorization or isolation.

2.2 Why hierarchy needs a scope system

The command tree from Chapter 1 answers where a command lives. It does not yet answer which configuration applies at that node. In `atlas`, a root-level `--config` flag should be available to every descendant. The `project` subtree may define a default region. The `project add` command may override the output format. A root output buffer installed by a test should capture messages written by a child. A help template configured on a subtree should affect that subtree without rewriting every leaf.

These are scope questions. A scope system lets a value be declared at one node and consumed at another according to a resolution law. Cobra repeatedly uses the same law: check the current command first; if the value is absent, walk toward the root; eventually use a process default when no command defines it.

The design is analogous to lexical scope in programming languages, but not identical. A command tree is an interface hierarchy rather than a syntax tree for one expression. Its inherited values include flags, streams, help functions, templates, error prefixes, and normalization behavior.

2.3 Local environments as partial maps

We begin with a mathematical model that is more general than flags.

Definition 2.1 (Policy key and policy value). Let K be a set of policy keys and W a set of policy values. Examples are:

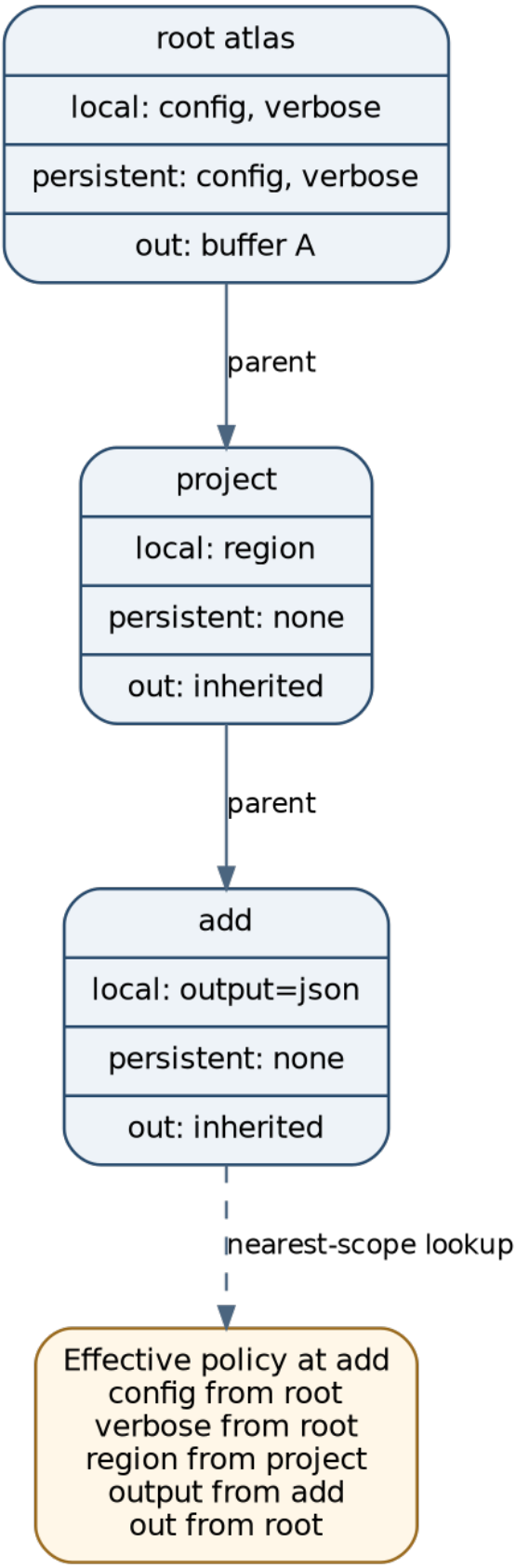

Figure 2.1: Nearest-scope policy lookup at the `add` command.

```
key                       possible value
-----------------------------------------------------------
flag:config               a flag declaration
stream:stdout             an io.Writer
template:help             a template function
error:prefix              a string
normalization:function    a name-normalization function
```

Different keys may have different value types in an implementation. The mathematical presentation treats them uniformly to explain lookup.

Definition 2.2 (Local environment). Each command node v has a finite partial map

$$L_v : K \rightharpoonup W,$$

called its local environment. $L_v(k)$ is defined when the node locally declares policy key k.

A partial map is appropriate because most nodes define only a few policies. The `project add` node may define a local `output` flag but no local output stream.

> **Fundamentals - Environments and lexical scope.** An environment maps names to meanings. In a programming-language interpreter, an environment might map variable names to values or memory locations. Entering a nested scope extends the environment. A local declaration can shadow an outer declaration. Command policy inheritance uses the same abstract mechanism even though the names denote flags, streams, or functions rather than ordinary variables.

2.4 The override operator

To combine environments, we need an operator that gives one map priority over another.

Definition 2.3 (Left-biased override). For partial maps $A, B : K \rightharpoonup W$, define $A \rhd B$ by

$$(A \rhd B)(k) = \begin{cases} A(k), & \text{if } A(k) \text{ is defined,} \\ B(k), & \text{otherwise.} \end{cases}$$

The left map wins. Thus $L_{\text{child}} \rhd L_{\text{parent}}$ expresses local shadowing.

The operator has useful algebraic properties:

1. **Associativity**:
$$(A \rhd B) \rhd C = A \rhd (B \rhd C).$$
2. **Identity**: the empty map $\emptyset$ satisfies
$$A \rhd \emptyset = \emptyset \rhd A = A.$$
3. **Idempotence**:
$$A \rhd A = A.$$
4. **Non-commutativity in general**:
$$A \rhd B \neq B \rhd A$$
when both define the same key differently.

Associativity means an ancestry chain can be folded without ambiguity. Non-commutativity is the mathematical form of precedence.

Side topic - A monoid of scoped maps. Partial maps under left-biased override and the empty map form a monoid: the operation is associative and has an identity. This gives a compact way to reason about folding local environments along a path. The operator is not commutative because scope order matters.

2.5 Effective policy and nearest-scope lookup

Let the path from root to node v be

$$r = v_0, v_1, \dots, v_n = v.$$

Definition 2.4 (Effective environment). The effective environment at v is

$$\Gamma_v = L_{v_n} \rhd L_{v_{n-1}} \rhd \cdots \rhd L_{v_0} \rhd B,$$

where B is a base environment of process or framework fallbacks. The Greek letter Γ is conventional for an environment and avoids confusing effective policy with the edge relation E from Chapter 1.

The rightmost map has the lowest precedence. Equivalently, lookup can be defined recursively.

Definition 2.5 (Nearest-scope lookup). For key k,

$$\text{lookup}(v, k) = \begin{cases} L_v(k), & L_v(k) \text{ is defined,} \\ \text{lookup}(\text{parent}(v), k), & v \neq r, \\ B(k), & v = r \text{ and } L_r(k) \text{ is undefined.} \end{cases}$$

This is the law summarized informally as "local first, then nearest ancestor, then default."

Definition 2.6 (Shadowing). A local declaration $L_v(k)$ shadows an ancestor declaration $L_u(k)$ when u is an ancestor of v and no node strictly between u and v declares k. The effective value at v is the local declaration.

Definition 2.7 (Provenance). The provenance of an effective value is the nearest node at which the winning declaration occurs, or the fallback source if no command node declares it.

Provenance is not cosmetic. Help and diagnostics can explain whether a flag is local or inherited. Tests can ensure a subtree does not accidentally inherit a policy. Documentation can separate local options from global options.

2.6 Worked example 1: effective flags at `project add`

Suppose the running example declares:

- root persistent flags: `config`, `verbose`, `output=table`;
- `project` persistent flag: `region=us-east`;
- `project add` local flag: `output=json`;
- no other flag declarations.

Represent the local environments as:

$$L_r = \{\texttt{config} \mapsto C, \texttt{verbose} \mapsto V, \texttt{output} \mapsto O_T\},$$

$$L_{project} = \{\texttt{region} \mapsto R\},$$

$$L_{add} = \{\texttt{output} \mapsto O_J\}.$$

Then

$$\Gamma_{add} = L_{add} \rhd L_{project} \rhd L_r.$$

Evaluating key by key gives:

Key	Winning declaration	Effective value	Provenance
`config`	root	`C`	inherited from root
`verbose`	root	`V`	inherited from root
`region`	`project`	`R`	inherited from parent
`output`	`add`	`O_J`	local; root value shadowed

The important distinction is that the root `output` flag is not merely overwritten in a flat map. It is excluded from the inherited view because the child owns that name locally.

A Cobra-like API exposes three useful projections:

```
func (c *Command) LocalFlags() *FlagSet
func (c *Command) PersistentFlags() *FlagSet
func (c *Command) InheritedFlags() *FlagSet
```

The effective parser sees a merged set, while help and docs can preserve provenance.

2.7 Persistent declarations versus local declarations

Cobra flags distinguish two declaration modes.

Definition 2.8 (Local flag). A local flag belongs to one command's own option namespace and does not automatically flow to descendants.

Definition 2.9 (Persistent flag). A persistent flag is declared on one command and is eligible to flow to descendants through the ancestry relation.

The word "persistent" here means persistent across the command subtree, not durable across program executions.

For a descendant v, the inherited flag set is conceptually

$$I_v = \left(\bigcup_{u \in \text{ancestors}(v)} P_u \right) \setminus \text{dom}(L_v),$$

with collision and normalization rules applied. The set notation is only approximate because order and object identity matter in an implementation; the key idea is that locally owned names are removed from the inherited projection.

2.7.1 Why separate local, inherited, and effective views?

A flattened effective map answers "what can the user set?" It does not answer:

- where was the option declared?
- is the option intended for this command or merely globally available?

- which declaration will documentation edit?
- did a child intentionally shadow an ancestor?

A mature scope system exposes both values and provenance. This principle generalizes beyond CLIs to configuration overlays, CSS cascades, dependency-injection containers, and nested policy systems.

2.8 Worked example 2: stream inheritance

Flags are not the only inherited policy. Consider output streams.

A test constructs the root and attaches a buffer:

```
buf := new(bytes.Buffer)
root.SetOut(buf)
root.SetErr(buf)
root.SetArgs([]string{"project", "list"})
err := root.Execute()
```

The selected child calls

```
fmt.Fprintln(cmd.OutOrStdout(), "mars")
```

If the child has no local output writer, stream lookup follows the parent chain. The buffer on the root becomes the effective writer at the child.

Mathematically, with key $k =$ `stream:stdout`,

$$L_{list}(k) \text{ undefined}, \quad L_{project}(k) \text{ undefined}, \quad L_r(k) = \text{buf},$$

so

$$\Gamma_{list}(k) = \text{buf}.$$

This example shows that inheritance shares an object. The buffer is not cloned. If several children write concurrently, the writer's own concurrency semantics matter.

2.9 Worked example 3: subtree-specific help policy

Suppose `atlas project` wants a specialized help format while `atlas cache` uses the root format. Place the override on `project`:

```
project.SetHelpFunc(projectHelp)
```

Then any descendant of `project` without a local help function resolves to `projectHelp`; the `cache` subtree continues to resolve to the root or default help function.

This is a useful middle ground between global configuration and per-leaf repetition. The subtree becomes a policy boundary.

2.10 The scope law and its proof

We can state a basic correctness property.

Proposition 2.1 (Nearest declaration wins). Let u be the nearest ancestor of v, possibly $u = v$, such that $L_u(k)$ is defined. Then $\Gamma_v(k) = L_u(k)$.

Proof. Write the path from v upward. By definition, every node between v and u has no local binding for k. Left-biased override therefore passes through those maps. At u, the binding is defined, so the override selects it and ignores all lower-precedence ancestor/default maps for key k. □

The proof is simple, but naming the proposition helps when implementing caches or alternate traversal algorithms. Any optimized implementation must preserve the same result.

2.11 Accidental scope widening

Inheritance reduces repetition, but broad scope is not always desirable. A root persistent flag becomes part of every descendant's effective namespace. This can introduce three problems:

1. **Name collision.** A descendant wants a local flag with the same name.
2. **False affordance.** Help suggests a global flag on a command that cannot meaningfully use it.
3. **Hidden coupling.** A new root-level policy silently changes many leaves.

A useful design question is not only "can this value be inherited?" but "what is the smallest subtree that should own the default?"

Definition 2.10 (Scope of a declaration). The scope of a declaration at node u is the set

$$\text{scope}(u) = \{v \in V \mid u \text{ is an ancestor of } v\},$$

possibly restricted by declaration kind and shadowing. Moving a declaration upward weakly enlarges its potential scope.

This gives a review rule: when a policy is moved toward the root, treat it as an API expansion, not a refactor with no semantics.

2.12 Late-bound framework defaults

Inheritance handles values already declared on the tree. Frameworks also want to supply behavior that the application did not declare: a help flag, version flag, help command, or completion command.

Eagerly adding every default at object construction is tempting, but it can steal names, change `HasSubCommands`, alter generated docs, and prevent application overrides. Cobra instead synthesizes many defaults at the latest practical phase and only when the semantic slot is unclaimed.

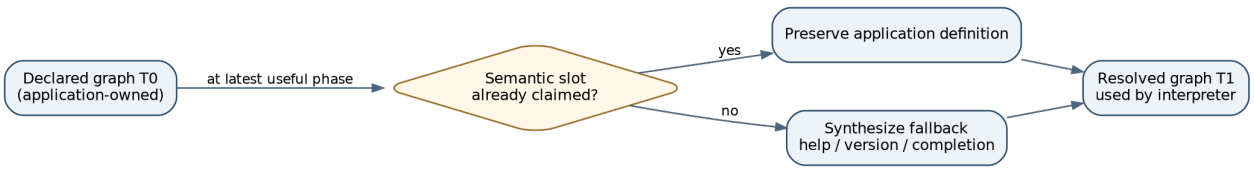

Figure 2.2: Late default synthesis preserves an application-owned definition and adds a fallback only when the slot is empty.

Definition 2.11 (Semantic slot). A semantic slot is a role in the model recognized by the framework, such as "the help flag," "the version flag," or "the help command." A slot may be identified by a stable name, annotation, interface, or explicit field.

Definition 2.12 (Late synthesized default). Let T_0 be an application-declared model and let $\mathcal{D}_p$ be the default-synthesis operator for phase p. A late synthesized default produces

$$T_1 = \mathcal{D}_p(T_0)$$

where $\mathcal{D}_p$ adds a framework definition only if the corresponding semantic slot is unclaimed in T_0.

A well-behaved default operator should satisfy several laws.

2.12.1 Preservation law

If the application has claimed slot s, default synthesis preserves that definition:

$$\text{claimed}(T, s) \implies \text{definition}(\mathcal{D}_p(T), s) = \text{definition}(T, s).$$

2.12.2 Idempotence law

Applying synthesis twice should not create duplicates:

$$\mathcal{D}_p(\mathcal{D}_p(T)) = \mathcal{D}_p(T).$$

2.12.3 Minimal-perturbation law

If a default is irrelevant to phase p, it should not alter observable properties needed before that phase. Cobra's temporary completion command is motivated by this law: retaining it in an ordinary root-only invocation would change whether the root appears to have subcommands.

2.12.4 Phase-visibility law

Tools must state whether they inspect T_0 or the resolved T_1. A documentation generator that omits synthesized help commands may be correct for the declaration phase but not for the user-visible runtime phase.

2.13 Worked example 4: a user-owned version flag

Suppose `atlas` already defines `-v` for verbose output. The application also sets a version string. A naive framework might add `--version, -v`, creating a shorthand collision.

A late default algorithm can be written as:

```
install_version_default(command):
    if command.version is empty:
        return
    if effective flags already contain "version":
        return
    if shorthand "v" is free:
        add flag --version with shorthand -v
    else:
        add flag --version without shorthand
```

The default adapts to the existing namespace rather than treating its preferred shorthand as privileged.

2.14 Worked example 5: temporary completion topology

Suppose `atlas` has no user-visible subcommands in a small deployment. The completion system wants an internal `__complete` command so the shell can query the executable. If the framework installs that command permanently, the root now has a child, which may change help and routing behavior.

A more careful algorithm is:

```
1. Temporarily attach __complete.
2. Resolve the current argv.
3. If the invocation selected __complete, retain it for this query.
4. Otherwise remove it before ordinary execution continues.
```

The internal protocol exists only in the phase that needs it.

2.15 Counterexample 1: process-global configuration bypasses hierarchy

Cobra also contains package-level switches and registries, such as prefix matching, case sensitivity, persistent-hook traversal, template functions, initializer/finalizer lists, and a completion-function registry. These values are outside the command tree.

Formally, a process-global map G is consulted by all trees:

$$E_v(k) = G(k)$$

for global key k, regardless of ancestry. A child cannot shadow it, and two command trees in the same process cannot independently choose different values.

This may be justified by compatibility, but it is a different architecture. Reusers should prefer graph-owned or executor-owned configuration when instance isolation matters.

2.16 Counterexample 2: inheritance is not authorization

Suppose a root command stores an authenticated administrator principal in context, and every child inherits it. It is tempting to treat "inherited principal exists" as permission. That is unsafe. Identity propagation and authorization are different functions:

$$\text{principal}(v) = \text{alice}$$

does not imply

$$\text{may}(\text{alice}, \texttt{cache clear --all}) = \text{true}.$$

High-risk authority should be checked explicitly near the effect boundary, even if identity or policy inputs are inherited.

> **Warning - Scope is not trust.** Nearest-scope lookup is a resolution rule. It does not establish that the winning value is authentic, authorized, fresh, or safe. Configuration inheritance and security policy may use similar shapes while protecting very different invariants.

2.17 Counterexample 3: mutable shared values

If a root provides a mutable formatter object and descendants inherit the same pointer, a leaf that mutates it changes behavior for siblings. The environment model resolves a value; it does not promise copying or immutability.

A robust design must state one of the following:

- inherited values are immutable;
- descendants receive copies;
- mutation is synchronized and intentionally shared;
- ownership is transferred rather than shared.

2.18 Caching effective environments

Repeated parent walks can be optimized by caching derived environments. However, cache correctness depends on mutation.

Let C_v be a cached effective environment. If an ancestor u changes L_u, then every descendant $v \in \text{scope}(u)$ may have a stale cache. Invalidation cost is proportional to the affected subtree unless versions or lazy recomputation are used.

A versioned scheme might assign each node a local version and compute a path digest:

$$\text{stamp}(v) = H(\text{stamp}(\text{parent}(v)), \text{version}(L_v)).$$

The cache is valid only when its stored stamp equals the current path stamp. This is not Cobra's exact mechanism; it is a reusable way to reason about derived scope caches.

2.19 Design checklist

For each inherited policy, ask:

- What is the key and value type?
- What is the declaration scope?
- Does local shadowing exist?
- Can tooling recover provenance?
- Are inherited values immutable, copied, or shared?
- What process-global values bypass the hierarchy?
- Which defaults are synthesized, in what phase, and with what presence check?
- Is default synthesis idempotent and minimally perturbing?
- Which values are configuration, and which require explicit authorization checks?

2.20 Chapter summary

A command hierarchy becomes a scope system when nodes carry local environments and descendants resolve values by nearest-scope lookup. Left-biased override formalizes shadowing, provenance distinguishes local from inherited values, and persistent declarations define subtree-wide policy. Late default synthesis adds framework convenience without outranking application intent, provided preservation, idempotence, and phase-visibility laws hold. Inheritance reduces repetition but can widen scope, share mutable state, and obscure security boundaries. Process-global configuration lies outside the hierarchy and should be treated as a separate compatibility tradeoff.

Source correspondence. The concrete evidence for this chapter is concentrated in `command.go` for flags, stream/template inheritance, and default synthesis, plus `command_test.go` and `completions.go` for shadowing and phase-sensitive behavior.

2.21 Exercises

2.21.1 Algebra and proofs

1. **Associativity.** Prove that left-biased override is associative by considering an arbitrary key k and the cases in which $A(k)$, $B(k)$, and $C(k)$ are defined.
2. **Non-commutativity.** Give the smallest pair of partial maps A, B for which $A \triangleright B \neq B \triangleright A$.
3. **Nearest declaration.** Prove Proposition 2.1 by induction on the distance from v to the nearest declaring ancestor.
4. **Idempotent defaults.** Give sufficient conditions on a synthesis operator $\mathcal{D}_p$ to prove $\mathcal{D}_p(\mathcal{D}_p(T)) = \mathcal{D}_p(T)$.

2.21.2 Worked scope calculations

5. The root declares persistent flags `config`, `output=table`, and `verbose`. `project` declares persistent `output=yaml` and local `region`. `project add` declares local `output=json`. Compute the local, inherited, and effective flag views at each node. State which declarations are shadowed.
6. A root sets stdout to buffer A. `project` sets stdout to buffer B. `project add` has no local writer. `cache clear` sets stdout to buffer C. Determine the effective writer at all four nodes and list the provenance.
7. Move `region` from `project` to the root. How does scope(`region`) change? Give one benefit and one risk.

2.21.3 Design and counterexamples

8. **Explicit authority handoff.** Redesign an inherited `principal` value so that each effectful leaf must explicitly request an authorization decision. Provide API signatures and a short execution trace.
9. **Cache invalidation.** Design a cache for effective environments under dynamic command insertion. State the invalidation rule and its worst-case cost.
10. **Phase mismatch.** Construct a case in which generated docs inspect the declaration phase while runtime users observe the default-resolved phase. What discrepancy appears, and how would you repair the build?
11. **Global switch isolation.** Design an executor-scoped alternative to a package-global `EnablePrefixMatching` switch. Show how two command trees in one process can use different policies.
12. **Mutable inherited value.** Give a concrete bug caused by a descendant mutating an inherited object. Repair it once with immutability and once with copying. Compare the costs.

Chapter 3

Operational Semantics of Command Execution

3.1 Learning objectives

By the end of this chapter, you should be able to:

- model execution as a small-step transition system;
- identify the admission frontier and its predicate;
- compose positional validators while preserving diagnostic order;
- derive root-to-leaf and leaf-to-root hook sequences;
- distinguish lifecycle post-stages from guaranteed cleanup;
- classify resolution, parse, admission, effect, and post-stage errors.

3.2 Why a command is more than a callback

A naive command framework maps a name directly to a function:

```
"add" -> addProject(args)
```

A mature framework must do more before the function is allowed to act. It must resolve the command, parse flags in the right scope, handle help and version requests, validate positional arguments, run preparation hooks, check required flags and cross-flag constraints, propagate context, select output channels, and classify errors. The ordering of these steps is observable. Moving validation after the effect can turn a harmless input error into a partial state change.

Cobra makes the order visible in its root executor and selected-command executor. This chapter models that order as an operational semantics: a transition system over machine configurations. The goal is not formal verification of the entire library. The goal is a precise vocabulary for questions such as:

- What must be true before `RunE` begins?
- Which hooks run on which paths?
- What kind of error occurred?
- Which cleanup actions are guaranteed?
- How do validators compose?

3.3 Invocation state

We need a state rich enough to describe execution without copying every implementation field.

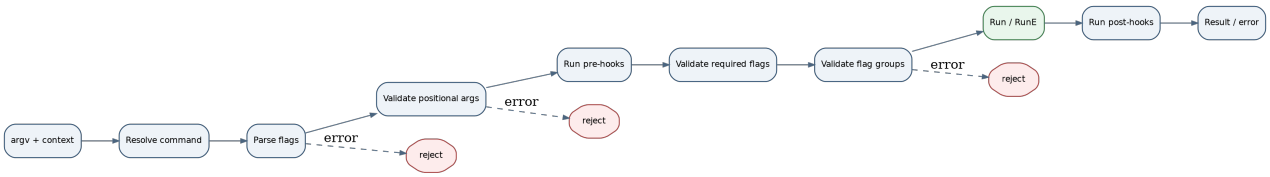

Figure 3.1: The command execution pipeline. The effect stage is reached only after earlier admission stages succeed.

Definition 3.1 (Invocation configuration). An invocation configuration is a tuple

$$\kappa = (T, v, a, F, C, S, q, o),$$

where:

- T is the command graph;
- v is the currently selected command node or the root during resolution;
- a is the remaining token sequence;
- F is parsed flag state;
- C is execution context and inherited policy;
- S is application-visible mutable state or effect environment;
- q is the current execution phase;
- o is accumulated observable output, diagnostics, and error information.

The exact shape of S depends on the application. For `atlas`, it might include a project service, cache store, and filesystem. Keeping S abstract lets us separate framework admission from domain effects.

Definition 3.2 (Execution phase). Let the phase set be

$$Q = \{\mathsf{Resolve}, \mathsf{Parse}, \mathsf{ArgCheck}, \mathsf{Pre}, \mathsf{FlagCheck}, \mathsf{Effect}, \mathsf{Post}, \mathsf{Done}, \mathsf{Failed}\}.$$

A real implementation has more detailed substages, such as default initialization and help/version short circuits. This abstraction preserves the important order.

Definition 3.3 (Small-step transition). The relation

$$\kappa \longrightarrow \kappa'$$

means the executor can take one legal step from configuration κ to configuration κ'. A complete execution is a trace

$$\kappa_0 \longrightarrow \kappa_1 \longrightarrow \cdots \longrightarrow \kappa_n$$

ending in phase Done or Failed.

> **Fundamentals - Small-step and big-step semantics.** Small-step semantics describes individual state transitions and is useful for ordering, interruption, and intermediate invariants. Big-step semantics relates an initial configuration directly to a final result. Framework lifecycle questions are often easier in small-step form because the skipped and executed phases are explicit.

3.4 The abstract transition rules

The central successful transitions can be written as inference rules. Let ok_X mean stage X succeeds.

3.4.1 Resolution

$$\frac{\mathrm{resolve}_T(a) = (v, a')}{(T, r, a, F, C, S, \mathsf{Resolve}, o) \longrightarrow (T, v, a', F, C, S, \mathsf{Parse}, o)}$$

3.4.2 Flag parsing

$$\frac{\mathrm{parse}(v, a') = (F, a'')}{(T, v, a', \emptyset, C, S, \mathsf{Parse}, o) \longrightarrow (T, v, a'', F, C, S, \mathsf{ArgCheck}, o)}$$

3.4.3 Positional argument validation

$$\frac{\mathrm{argsValid}(v, a'')}{(T, v, a'', F, C, S, \mathsf{ArgCheck}, o) \longrightarrow (T, v, a'', F, C, S, \mathsf{Pre}, o)}$$

3.4.4 Pre-hooks

$$\frac{\mathrm{preHooks}(v, C, S) = (C', S')}{(T, v, a'', F, C, S, \mathsf{Pre}, o) \longrightarrow (T, v, a'', F, C', S', \mathsf{FlagCheck}, o)}$$

3.4.5 Flag constraints

$$\frac{\mathrm{requiredValid}(v, F) \wedge \mathrm{groupsValid}(v, F)}{(T, v, a'', F, C', S', \mathsf{FlagCheck}, o) \longrightarrow (T, v, a'', F, C', S', \mathsf{Effect}, o)}$$

3.4.6 Main effect

$$\frac{\mathrm{run}(v, a'', F, C', S') = (S'', o')}{(T, v, a'', F, C', S', \mathsf{Effect}, o) \longrightarrow (T, v, a'', F, C', S'', \mathsf{Post}, o \cdot o')}$$

3.4.7 Post-hooks and completion

$$\frac{\mathrm{postHooks}(v, C', S'') = (C'', S''')}{(T, v, a'', F, C', S'', \mathsf{Post}, o) \longrightarrow (T, v, a'', F, C'', S''', \mathsf{Done}, o)}$$

Each stage also has a failure rule that transitions directly to Failed with an error. The absence of a transition through later phases is exactly what makes hook guarantees nontrivial.

3.5 Effect admission

The most important property of the pipeline is that the domain effect has preconditions.

Definition 3.4 (Admission predicate). For selected command v, arguments a, parsed flags F, and context C, define

$$\mathrm{Admit}(v, a, F, C) = R(v) \wedge A(v, a) \wedge P(v, F) \wedge G(v, F) \wedge H(v, C),$$

where:

- $R(v)$ means the command is runnable;
- $A(v, a)$ means positional arguments are valid;
- $P(v, F)$ means required flags are present;
- $G(v, F)$ means flag-group constraints hold;
- $H(v, C)$ means pre-hook preparation succeeded.

Parsing success and command resolution are assumed before the predicate is evaluated; they can also be included explicitly.

Proposition 3.1 (Effect-admission safety). In the abstract pipeline, if phase Effect is reached, then $\text{Admit}(v, a, F, C)$ holds for the values produced by prior phases.

Proof sketch. The only successful transition into Effect comes from FlagCheck. Reaching FlagCheck requires successful resolution, parsing, argument validation, and pre-hooks. The transition rule from FlagCheck requires required-flag and group validation. Therefore every conjunct holds. □

This proposition is a framework guarantee about *admission order*. It does not prove that the domain handler is correct or authorized.

> **Side topic - Hoare triples.** A Hoare triple $\{P\}\ c\ \{Q\}$ says that if precondition P holds and command c terminates, postcondition Q holds. The execution pipeline can be read as establishing a precondition for RunE: $\{\text{Admit}\}$ RunE $\{\text{handler-specific result}\}$. The framework establishes P; the application is responsible for Q.

3.6 Worked example 1: a successful trace

Consider

```
atlas --config dev.yaml project add mars --region us-east
```

Assume:

- `project add` requires exactly one positional argument;
- `config` is a root persistent flag;
- `region` is visible at `add`;
- pre-hooks load configuration and construct a project service;
- all flag constraints hold.

A high-level trace is:

Phase	Important state	Result
Resolve	command word `project add`	select node `add`
Parse	`config=dev.yaml`, `region=us-east`; args `mars`	success
ArgCheck	`len(args)=1`	success
Pre	load `dev.yaml`; attach service to context	success
FlagCheck	required/group predicates	success
Effect	call `addProject("mars", "us-east")`	project created
Post	emit summary or metrics	success
Done	returned error `nil`	terminal success

The domain effect begins only after the invocation has a selected command, parsed assignment, valid arity, and prepared service.

3.7 Worked example 2: a rejected trace

Now consider

```
atlas project add
```

The selected command is still `add`, but `ExactArgs(1)` fails. The trace ends at `ArgCheck`:

```
Resolve -> Parse -> ArgCheck -> Failed
```

Neither pre-hooks nor the domain effect run. This distinction matters if pre-hooks perform network calls or acquire resources.

3.8 Positional validators as predicates

Cobra represents positional validation with a function type similar to:

```
type PositionalArgs func(cmd *Command, args []string) error
```

Mathematically, a validator is a decidable predicate with an explanatory error:

$$V : V \times \Sigma^* \to \{\mathsf{valid}\} \cup \mathrm{Error}\,.$$

Ignoring error messages for a moment, examples are:

3.8.1 No arguments

$$V_0(a) \iff |a| = 0.$$

3.8.2 Exactly n arguments

$$V_{=n}(a) \iff |a| = n.$$

3.8.3 At least n arguments

$$V_{\geq n}(a) \iff |a| \geq n.$$

3.8.4 Range

$$V_{[m,n]}(a) \iff m \leq |a| \leq n.$$

3.8.5 Membership in a valid set

For finite S,

$$V_S(a) \iff \forall x \in a,\ x \in S.$$

Definition 3.5 (Validator conjunction). Given validators $V_1, \ldots, V_k$, their short-circuit conjunction is

$$\mathrm{All}(V_1, \ldots, V_k)(a) = \begin{cases} \text{first error produced by } V_i, & \text{if some } V_i \text{ fails,} \\ \mathsf{valid}, & \text{otherwise.} \end{cases}$$

Cobra's `MatchAll` has this shape. It is logical conjunction plus deterministic error selection by order.

3.9 Worked example 3: composing validators

Suppose `atlas cache clear` accepts exactly one key from a fixed set when `--all` is not used. Ignoring the flag dependency for the moment, positional validation can be:

```
Args: cobra.MatchAll(
    cobra.ExactArgs(1),
    cobra.OnlyValidArgs,
),
ValidArgs: []string{"projects", "templates", "sessions"},
```

The mathematical predicate is

$$|a| = 1 \land a_1 \in \{\texttt{projects}, \texttt{templates}, \texttt{sessions}\}.$$

Order affects the error message. Checking exact arity first avoids reporting an invalid element when there is no element at all.

> **Fundamentals - Logic versus diagnostics.** Boolean conjunction is commutative: $P \land Q = Q \land P$. Short-circuit validators with errors are not observationally commutative because the first failing validator determines the message. A framework must distinguish logical equivalence from diagnostic equivalence.

3.10 Flags as an assignment

Let a command expose flag variables $x_1, \dots, x_n$. Parsed flag state is an assignment

$$\alpha : \{x_1, \dots, x_n\} \to \text{Value}\,.$$

For presence-based constraints, define Boolean variables

$$b_i = \begin{cases} 1, & \text{if flag } x_i \text{ was explicitly set,} \\ 0, & \text{otherwise.} \end{cases}$$

A required flag x_i imposes $b_i = 1$. Group constraints are Boolean formulas; Chapter 4 will study how the same formulas drive both validation and completion.

3.11 Hook sequences

Lifecycle hooks are functions around the main effect. In a hierarchy, their order must be specified.

Let the ancestry path of selected node v be

$$r = v_0, v_1, \dots, v_n = v.$$

With full persistent-hook traversal enabled, the intended orders are:

$$\text{preSeq}(v) = \langle \text{pre}(v_0), \text{pre}(v_1), \dots, \text{pre}(v_n) \rangle,$$

$$\text{postSeq}(v) = \langle \text{post}(v_n), \text{post}(v_{n-1}), \dots, \text{post}(v_0) \rangle.$$

This mirrors stack discipline: enter broad scopes from root to leaf; leave narrow scopes from leaf to root.

Cobra also supports a compatibility mode in which only the first applicable persistent hook is used. The global switch that selects semantics is itself an architectural tradeoff: hook policy is process-wide rather than executor-specific.

3.12 Worked example 4: resource preparation

Suppose:

- root persistent pre-hook loads config;
- `project` persistent pre-hook authenticates to a project service;
- `add` local pre-hook validates a template file;
- `add` handler creates the project;
- local post-hook prints a receipt;
- persistent post-hooks record subtree and application metrics.

With traversal enabled, the successful order is:

```
root load config
project authenticate
add validate template
add create project
add print receipt
project record project metric
root record application metric
```

The order is semantically meaningful. Authentication needs configuration; the effect needs authentication; narrow metrics should see the effect before broad metrics aggregate it.

3.13 The cleanup counterexample

A common mistake is to treat post-hooks as `finally` blocks. They are not.

Consider:

```
var lock Lock

cmd.PreRunE = func(cmd *cobra.Command, args []string) error {
    return lock.Acquire()
}

cmd.RunE = func(cmd *cobra.Command, args []string) error {
    return updateDatabase() // may fail
}

cmd.PostRunE = func(cmd *cobra.Command, args []string) error {
    return lock.Release()
}
```

If `updateDatabase` returns an error and the executor returns immediately, `PostRunE` is skipped. The lock remains held.

The correct effect-owned shape is:

```
cmd.RunE = func(cmd *cobra.Command, args []string) error {
    if err := lock.Acquire(); err != nil {
        return err
```

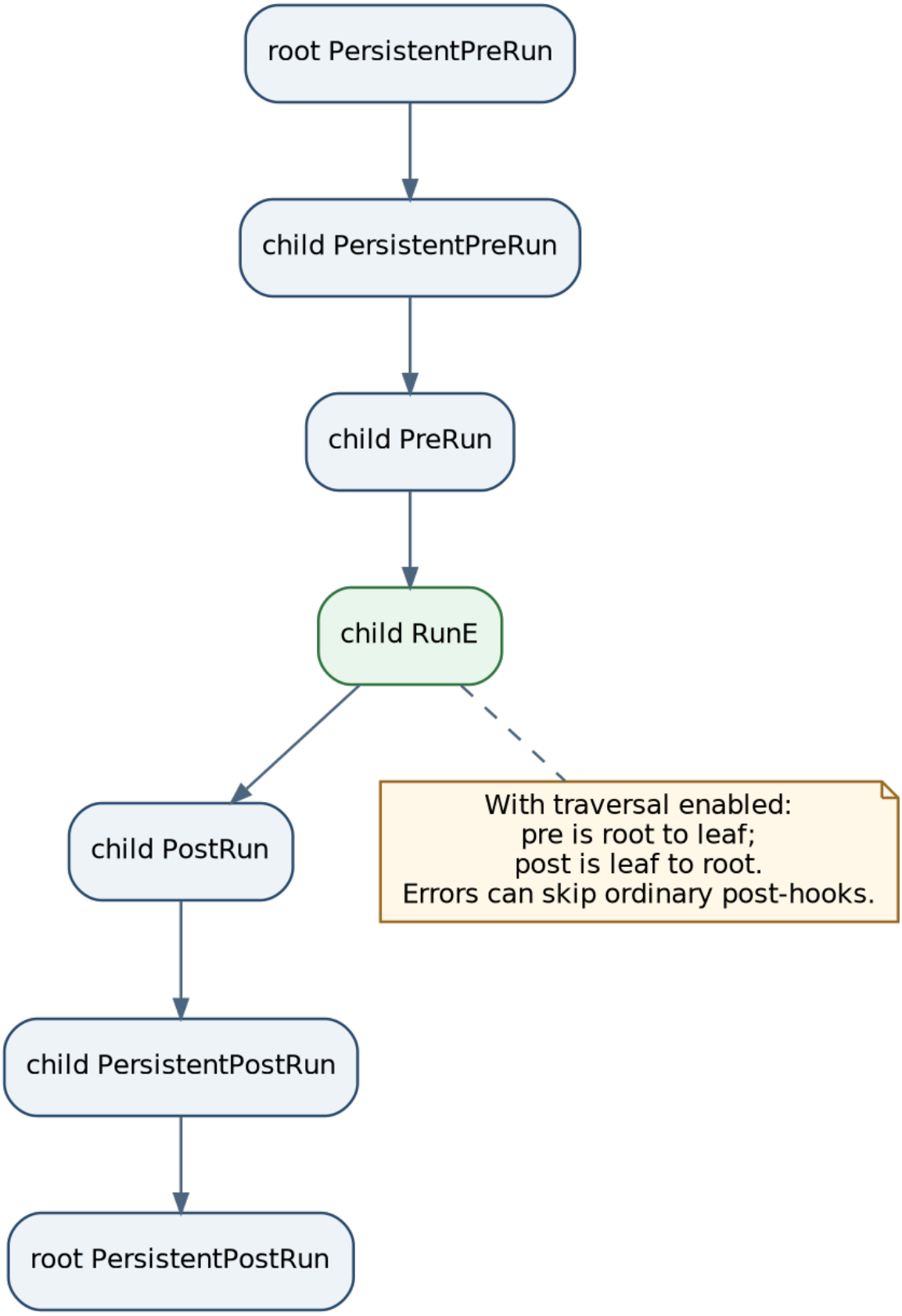

Figure 3.2: With persistent-hook traversal enabled, pre-hooks run root to leaf and post-hooks run leaf to root.

```
    }
    defer lock.Release()
    return updateDatabase()
}
```

The `defer` is installed in the same dynamic scope that acquired the resource.

Definition 3.6 (Guaranteed cleanup boundary). A cleanup boundary guarantees that once resource acquisition succeeds and cleanup registration completes, cleanup executes on every subsequent return path from the protected scope, including error returns and panics covered by the language mechanism.

Ordinary post-hooks do not satisfy this definition. They are later transition stages, not guaranteed unwinding actions.

Cobra's framework finalizers installed through a `defer` after framework initialization have stronger exit-path behavior than command post-hooks, but even that guarantee begins only after the defer is registered.

> **Warning - Lifecycle order is not unwinding semantics.** A stage named "post" communicates sequence, not guarantee. Whenever correctness depends on release, rollback, or restoration, identify the exact language or runtime construct that guarantees execution.

3.14 Pre-hooks before all validation?

In Cobra's order, positional argument validation occurs before pre-hooks, while required-flag and flag-group validation occur after pre-hooks. Therefore a pre-hook can run for an invocation that later fails flag validation.

This is neither automatically right nor wrong. It means pre-hooks should be designed with the remaining rejection points in mind.

A useful classification is:

- **pure preparation**: derive values, parse configuration, attach services;
- **reversible acquisition**: open a resource with guaranteed cleanup;
- **irreversible effect**: send a message, mutate a remote service, charge money.

Irreversible effects should normally occur only after all admission predicates that can be evaluated locally.

Definition 3.7 (Admission frontier). The admission frontier is the transition after which the framework considers the invocation eligible to perform the primary domain effect. In our model, it is the successful transition from `FlagCheck` to `Effect`.

Every effectful framework should name this frontier. Without it, "validation happens somewhere" is too weak to reason about partial effects.

3.15 Error classes

Different failures imply different operator and retry behavior.

Definition 3.8 (Resolution error). The token sequence does not denote a valid command path, or is ambiguous under the matching policy.

Definition 3.9 (Parse error). Tokens denote a command but cannot be parsed as flags and values under its effective schema.

Definition 3.10 (Admission error). Parsing succeeds, but arguments, required flags, group constraints, or preparation conditions reject the invocation before the main effect.

Definition 3.11 (Effect error). The main handler was entered and returned an error after domain work was attempted.

Definition 3.12 (Post-stage error). The main effect succeeded or at least returned normally, but a later post-hook failed.

These classes should not be flattened into one “command failed” category when retry or auditing matters. An admission error is usually safe to correct and retry; an effect error may have produced a partial external change.

3.16 Worked example 5: error taxonomy

Invocation / event	Class	Was main effect entered?	Retry concern
`atlas frobnicate`	resolution	no	fix command name
`atlas project add --region`	parse	no	provide flag value
`atlas project add`	admission	no	provide project name
`atlas project add mars --template missing` rejected by pre-hook	admission	no	repair file path
remote service creates project, then connection drops	effect	yes	check idempotency before retry
project created, receipt writer fails	post-stage	yes	do not assume effect rolled back

The last two cases show why “error means nothing happened” is unsafe after the admission frontier.

3.17 Context propagation

Execution context is part of the configuration C. A root execution can install caller-owned cancellation and deadlines. The selected child receives that context when it has no local context.

A handler should use it explicitly:

```
func addProject(cmd *cobra.Command, args []string) error {
    ctx := cmd.Context()
    return projectService.Add(ctx, args[0])
}
```

Context propagation is a channel, not a guarantee. If the service ignores `ctx`, cancellation remains advisory.

3.18 Disabling framework flag parsing

Some commands wrap another language or plugin and need raw tokens. `DisableFlagParsing` moves responsibility across the boundary.

Definition 3.13 (Parsing authority). Parsing authority is the component responsible for assigning syntactic roles and values to tokens. Normally Cobra owns it for flags. A command that disables parsing takes that authority into its handler or downstream parser.

This is useful for wrapper commands, but it weakens framework guarantees:

- required-flag validation may be inapplicable;
- grouped constraints may not be enforceable by the framework;
- completion must reason about both framework-known and externally parsed flags;
- error quality depends on the downstream parser.

An escape hatch should therefore state which admission laws are transferred to the application.

3.19 Determinism and ordering

A pipeline is easier to test when diagnostics are deterministic. Sorting missing flags or group identifiers ensures the first error does not depend on map iteration. Hook order tests ensure refactoring does not silently change lifecycle semantics.

Definition 3.14 (Trace determinism). For fixed model, argv, injected context, deterministic dependencies, and initial state, an executor is trace-deterministic when it produces the same sequence of framework phases and the same ordered diagnostics.

Trace determinism does not imply domain determinism. Network calls and clocks can still vary. It isolates framework behavior from incidental iteration order.

3.20 Design checklist

For an execution engine, write down:

- the configuration state;
- the phase set and transition order;
- the admission frontier;
- the exact predicate established before effects;
- hook order through a hierarchy;
- which errors skip which later stages;
- which cleanup mechanism has guaranteed unwinding semantics;
- which parser owns tokens after escape hatches;
- how resolution, parse, admission, effect, and post-stage errors are distinguished;
- which outputs are deterministic enough for scripts and tests.

3.21 Chapter summary

Command execution is an ordered state machine, not a direct callback lookup. A configuration records the graph, selected node, tokens, parsed assignment, context, application state, phase, and observations. Successful transitions establish an admission predicate before the main effect. Positional validators are predicates with diagnostic order; hierarchical hooks form ordered sequences; post-hooks are not guaranteed cleanup; and error classes reveal whether the effect was entered. Escape hatches such as disabled flag parsing transfer parsing authority and weaken framework-owned laws. Naming the admission frontier and cleanup boundary is essential for correctness.

> **Source correspondence.** The concrete evidence for this chapter is concentrated in `command.go`, `args.go`, and `command_test.go`, especially the ordered `ExecuteC`/`execute` path, validator combinators, context propagation, and hook-order tests.

3.22 Exercises

3.22.1 Formal semantics

1. **Failure rules.** Write the small-step failure rule for argument validation and the rule for an error returned by `RunE`. Which final configurations differ?
2. **Admission proof.** Expand Proposition 3.1 to include successful resolution and parsing as explicit conjuncts. Prove it by induction on trace length.
3. **Big-step relation.** Define a big-step relation $\langle T, a, S\rangle \Downarrow \langle S', o\rangle$ for successful execution. Explain what information about skipped hooks is lost compared with small-step semantics.
4. **Diagnostic non-commutativity.** Give two validators P and Q that are logically commutative but produce meaningfully different first errors under `MatchAll(P,Q)` and `MatchAll(Q,P)`.

3.22.2 Trace analysis

5. Trace

   ```
   atlas project add mars --template missing.yaml
   ```

   assuming resolution and parsing succeed, `ExactArgs(1)` succeeds, a pre-hook checks the template and fails, and the command has both local and persistent post-hooks. List exactly which stages run.

6. Trace a `RunE` failure under full persistent-hook traversal. Which ordinary post-hooks run? Which framework finalizers installed with `defer` may still run?

7. Design a hook stack for root configuration, subtree authentication, and leaf transaction management. State which resource each scope owns and how it is released on every error path.

3.22.3 Validators and APIs

8. Define `AtMostOneOfPositions(i,j,k)` as a positional validator over a token list. Then explain why some relations are better represented as flags rather than positional arguments.
9. Write a generic API for composing validators that can either stop at the first error or accumulate all errors. Compare usability and complexity.
10. Define a typed error hierarchy for resolution, parse, admission, effect, and post-stage errors. Show how a caller would decide whether an automatic retry is safe.

3.22.4 Counterexamples and design

11. Construct an invocation in which a pre-hook performs an irreversible effect and a later flag-group check fails. Explain the resulting inconsistency and redesign the phase order or hook responsibility.
12. Design an executor-scoped persistent-hook policy that replaces a process-global traversal switch. Include the type signature and a test for root-to-leaf/leaf-to-root order.
13. A wrapper command disables flag parsing and forwards to another program. List which framework guarantees are lost. Design a completion adapter that remains accurate without pretending Cobra owns the downstream grammar.
14. **Capstone trace property.** State and test the property “no domain write occurs before admission.” Describe the instrumentation required to detect writes during resolution, parsing, validation, and pre-hooks.

Chapter 4

Multiple Interpreters over One Model

4.1 Learning objectives

By the end of this chapter, you should be able to:

- distinguish authoritative, advisory, and generated interpreters;
- express common flag-group laws as Boolean formulas;
- reason about completion from partial assignments and legal extensions;
- specify a protocol-pure completion query and directive bit set;
- explain testability through injectable boundaries and observational equivalence;
- treat documentation generation as a fold with an injective artifact-name mapping.

4.2 Why users need more than rejection

A runtime validator can tell a user that an invocation is invalid. A good interface also helps the user avoid constructing the invalid invocation, explains the available grammar, offers context-sensitive candidates, supports deterministic tests, and publishes accurate reference documentation.

The dangerous implementation strategy is to build each of these features from an independent schema. The stronger strategy is to represent stable relations in the semantic model and write several interpreters:

- an authoritative interpreter rejects invalid invocations;
- an advisory interpreter guides partial input toward valid completion;
- a protocol interpreter exposes the live model to an external shell;
- a test interpreter substitutes controlled process boundaries;
- a documentation interpreter renders the assembled model.

The outputs differ, but their shared facts come from one authority.

4.3 Interpreters, projections, and authority

Chapter 1 defined an interpreter broadly. We now refine the roles.

Definition 4.1 (Authoritative interpreter). An interpreter is authoritative for a decision when its result controls whether the system admits or performs the corresponding effect. Runtime flag

validation is authoritative for the static flag-group laws Cobra enforces.

Definition 4.2 (Advisory interpreter). An interpreter is advisory when its result guides a user or tool but does not itself establish effect eligibility. Shell completion is advisory: a user can type tokens that were never suggested, and runtime validation must still decide legality.

Definition 4.3 (Projection). A projection extracts or derives one view from a model without becoming a second editable source of truth. Local flags, inherited flags, completion candidates, and generated reference pages are projections of the command graph.

A sound architecture can have several projections while preserving one authority. The important test is whether a projection is recomputed from the model or manually maintained beside it.

> **Warning - Assistance is not admission.** Completion can hide an illegal option and still be wrong due to stale dynamic state, user edits, or protocol failure. Never remove the authoritative runtime check merely because the UI "prevents" an invalid choice.

4.4 Constraint metadata

Cross-field constraints are an ideal example of one relation interpreted twice. Cobra supports three finite flag-group laws and stores group membership as annotations on the participating flags.

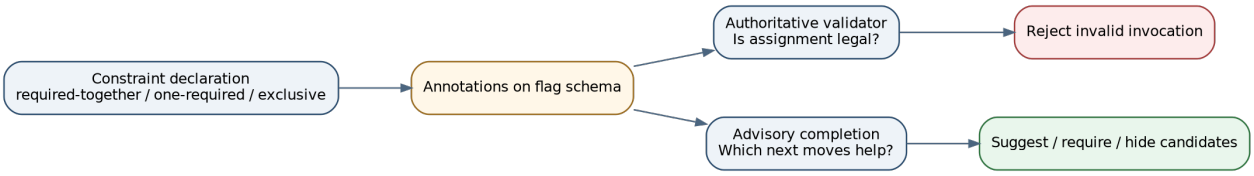

Figure 4.1: The same constraint metadata is interpreted by an authoritative validator and an advisory completion engine.

Let group $G = \{x_1, \ldots, x_n\}$ and presence variables $b_i \in \{0, 1\}$.

Definition 4.4 (Required-together constraint). If any member is present, every member must be present:

$$\left(\bigvee_{i=1}^{n} b_i \right) \Rightarrow \left(\bigwedge_{i=1}^{n} b_i \right).$$

Equivalent forms include "either none or all" and

$$\sum_{i=1}^{n} b_i \in \{0, n\}.$$

Definition 4.5 (One-required constraint). At least one member must be present:

$$\bigvee_{i=1}^{n} b_i,$$

or equivalently

$$\sum_{i=1}^{n} b_i \geq 1.$$

Definition 4.6 (Mutually-exclusive constraint). At most one member may be present:

$$\sum_{i=1}^{n} b_i \leq 1.$$

The declarations correspond to API shapes such as:

```
func (c *Command) MarkFlagsRequiredTogether(names ...string)
func (c *Command) MarkFlagsOneRequired(names ...string)
func (c *Command) MarkFlagsMutuallyExclusive(names ...string)
```

The annotation records the relation as data. Runtime validation reconstructs the Boolean assignment from `Changed` state and evaluates the formula.

4.5 Worked example 1: `cache clear`

The command

`atlas cache clear [--all | --key KEY]`

requires exactly one of `--all` and `--key`. Let

$$a = \mathbf{1}[\texttt{--all set}], \qquad k = \mathbf{1}[\texttt{--key set}].$$

The law is

$$a + k = 1.$$

Cobra's mutually-exclusive relation gives $a+k \leq 1$, while a one-required relation gives $a+k \geq 1$. Declaring both groups yields exact-one semantics.

Truth table:

all	key	one-required	exclusive	combined legal?
0	0	false	true	no
0	1	true	true	yes
1	0	true	true	yes
1	1	true	false	no

This example illustrates compositional constraint metadata: simple relations can be conjoined to express a stronger law.

4.6 Validation over complete assignments

Definition 4.7 (Complete assignment validator). Given a formula φ over presence variables and a complete assignment α, validation computes

$$\text{validate}(\varphi, \alpha) = \begin{cases} \textsf{success}, & \alpha \vDash \varphi, \\ \textsf{error}, & \text{otherwise.} \end{cases}$$

The notation $\alpha \vDash \varphi$ means assignment α satisfies formula φ.

Runtime validation is binary: the invocation is legal or not. Completion operates on partial assignments.

4.7 Completion over partial assignments

When the user has typed some flags but not others, the system knows a partial assignment ρ. Some variables are fixed; others are unassigned.

Definition 4.8 (Partial assignment and legal extension). A partial assignment is a function

$$\rho : \{x_1, \ldots, x_n\} \to \{0, 1, \bot\},$$

where $\bot$ means "not assigned yet." A complete assignment α is a legal extension of ρ for formula φ when

1. α agrees with every value already fixed by ρ; and
2. $\alpha \vDash \varphi$.

The set of legal extensions is

$$\text{Ext}(\rho, \varphi) = \{\alpha \mid \alpha \supseteq \rho \wedge \alpha \vDash \varphi\}.$$

Definition 4.9 (Admissible next variable). An unset flag x is admissible as a next positive choice when there exists some $\alpha \in \text{Ext}(\rho \cup \{x = 1\}, \varphi)$.

This definition generalizes completion filtering. Cobra's three group types permit simpler specialized logic:

- required-together: once one member is set, suggest the missing members as required;
- one-required: while none is set, suggest group members as required choices;
- mutually-exclusive: once one member is set, hide the remaining members.

These transformations are projections of the same formulas used by validation.

4.8 Worked example 2: completion states

For exact-one constraint on `all` and `key`:

4.8.1 No group flag typed

Partial assignment:

$$\rho_0 = \{a = \bot, k = \bot\}.$$

Legal next positive choices are `--all` and `--key`.

4.8.2 `--all` typed

$$\rho_1 = \{a = 1\}.$$

Every legal extension requires $k = 0$. Completion should hide `--key`.

4.8.3 `--key` typed

$$\rho_2 = \{k = 1\}.$$

Every legal extension requires $a = 0$. Completion should hide `--all` and proceed to complete a key value.

The validator and completer answer different questions:

```
validator: Is this finished assignment legal?
completer: Which next edits preserve at least one legal completion?
```

4.9 Deterministic diagnostics

Constraint data may be stored in maps or discovered in arbitrary order. Scripts and tests benefit when the first reported error is stable. Sorting group identifiers and missing flag names creates deterministic diagnostics.

Definition 4.10 (Diagnostic determinism). For fixed model and assignment, a validator is diagnostically deterministic when it returns the same ordered error representation independent of incidental map or traversal order.

This property is weaker than logical completeness. A validator may still report only the first violated formula, but it should choose that formula predictably.

4.10 Counterexample 1: duplicate dynamic checks

Suppose a required-together relation is encoded in annotations, but the handler also checks it manually with slightly different rules:

```
metadata: if --template is set, --region is required
handler:  if --template is set, --region and --owner are required
```

Completion guides the user to one “valid” state; runtime handler rejects it for another reason. There are two authorities.

The repair is not necessarily to encode every business rule as static metadata. The repair is to classify the rules:

- static structural rule: put it in schema metadata;
- dynamic rule depending on remote state, time, or authorization: keep it in the runtime authority and expose only a clearly advisory preview.

4.11 Counterexample 2: constraints requiring a solver

Consider:

$$(x \vee y) \wedge (z \Rightarrow x) \wedge (\neg y \vee w) \wedge (\text{exactly two of } x, y, z, w).$$

Simple “mark required” and “hide peer” mutations may not compute all legal next choices. A general completion engine would need a constraint solver over partial assignments. Metadata is still valuable, but the interpreter becomes more sophisticated.

> **Side topic - SAT and constraint satisfaction.** Boolean satisfiability asks whether some assignment makes a formula true. Completion under constraints often asks a related incremental question: after fixing the user’s current choices, which additional assignments leave the formula satisfiable? For simple flag groups, direct rules are clearer than a SAT solver. For rich forms or configuration languages, a solver may become appropriate.

4.12 The executable as a query server

Static shell scripts cannot know all dynamic command state. Cobra’s response is to let the shell query the executable through hidden commands such as `__complete` and `__completeNoDesc`.

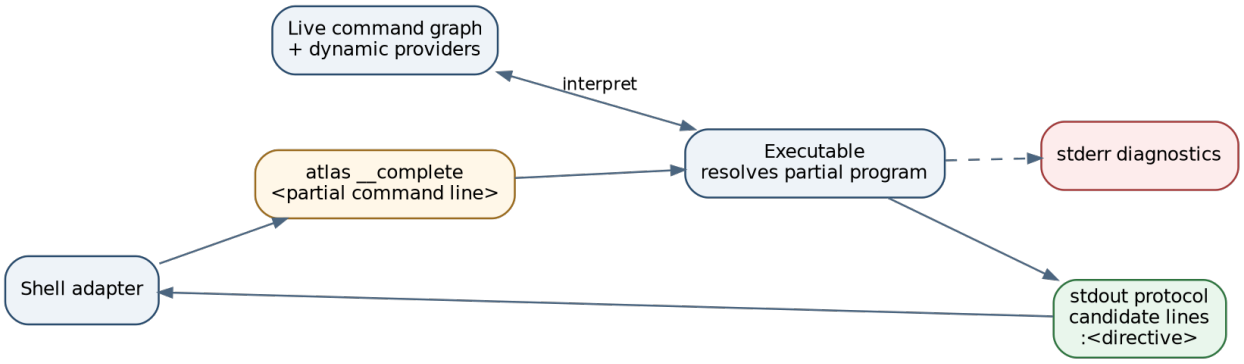

Figure 4.2: A shell adapter queries the executable, which interprets the partial command line against the live model and returns a strict stdout protocol.

The request contains a *partial program*: the user may be in the middle of a command name or flag value. Normal execution cannot simply be invoked, because incomplete input is expected rather than erroneous.

Definition 4.11 (Partial command program). A partial command program is a token sequence whose last token may be an incomplete prefix and whose flag/value structure may not yet form a complete executable invocation.

Definition 4.12 (Completion query). A completion query is a function

$$Q_C : T \times \text{PartialArgv} \times C \to \text{Candidates} \times \text{Directive} \times \text{Diagnostics}\,.$$

It resolves as much of the command path as possible, parses enough state to determine the completion position, applies static constraints, invokes dynamic providers, and returns shell-neutral guidance.

A Cobra-like dynamic completion signature is:

```
type CompletionFunc func(
    cmd *Command,
    args []string,
    toComplete string,
) ([]Completion, ShellCompDirective)
```

4.13 Completion directives as a bit set

Candidate strings are not enough. The shell needs to know whether to append a space, fall back to filename completion, filter by extension, restrict to directories, or preserve candidate order.

Let directive atoms be

$$D = \{d_{error}, d_{nospace}, d_{nofile}, d_{fileext}, d_{dirs}, d_{keeporder}\}.$$

A directive is a subset of D, represented efficiently as a bit map.

Definition 4.13 (Bit-set encoding). Assign each atom d_i a power of two 2^i. A directive set $S \subseteq D$ is encoded as

$$\text{code}(S) = \sum_{d_i \in S} 2^i.$$

Bitwise tests recover membership. The shell adapter does not need to understand internal command objects; it only interprets the protocol vocabulary.

4.14 Protocol purity

A typical response has one candidate per line and a final line of the form

```
:<integer directive>
```

Human diagnostics go to stderr so the shell can ignore them.

Definition 4.14 (Protocol-pure stdout). A command has protocol-pure stdout when every byte written to stdout belongs to the specified machine response grammar. Human logs, progress messages, and warnings must use another channel.

Protocol pollution is a correctness defect. A debug print can become a bogus completion candidate.

A simple response grammar is:

```
response   ::= candidate* directive-line
candidate  ::= text [TAB description] NEWLINE
directive-line ::= ":" integer NEWLINE
```

The grammar also motivates sanitization: a candidate description containing a newline must be truncated or escaped so it cannot create extra protocol records.

4.15 Side effects and latency in completion

Dynamic completion runs application code, often repeatedly while the user types. Therefore a completion function should be treated as a query interpreter with stricter operational expectations than a normal command:

- avoid writes and irreversible effects;
- avoid prompts;
- honor context cancellation;
- bound network latency;
- cache carefully if results are expensive;
- keep stdout protocol-pure;
- return useful fallback directives when no candidates exist.

Completion is not a security boundary. If listing remote resources requires authorization, the completion function must use the same policy source as runtime and must still assume the final invocation will be rechecked.

4.16 Worked example 3: dynamic project completion

Suppose `atlas project show NAME` completes project names from a service.

```
func completeProject(
    cmd *cobra.Command,
    args []string,
    prefix string,
) ([]cobra.Completion, cobra.ShellCompDirective) {
    names, err := projectService.ListNames(cmd.Context(), prefix)
    if err != nil {
        return nil, cobra.ShellCompDirectiveError |
            cobra.ShellCompDirectiveNoFileComp
    }
    return names, cobra.ShellCompDirectiveNoFileComp
}
```

Pedagogically, the important properties are:

1. the selected command and context come from the same graph/executor;
2. the function is a query, not an effect;
3. failure returns an explicit directive;
4. file completion is disabled because project names are not paths;
5. runtime `show` must still validate and authorize the chosen name.

4.17 Injectable process boundaries

A second interpreter-like use of the model is in-process testing. Rather than force every test through a subprocess, Cobra lets callers substitute arguments, context, and streams while running the same resolver and executor.

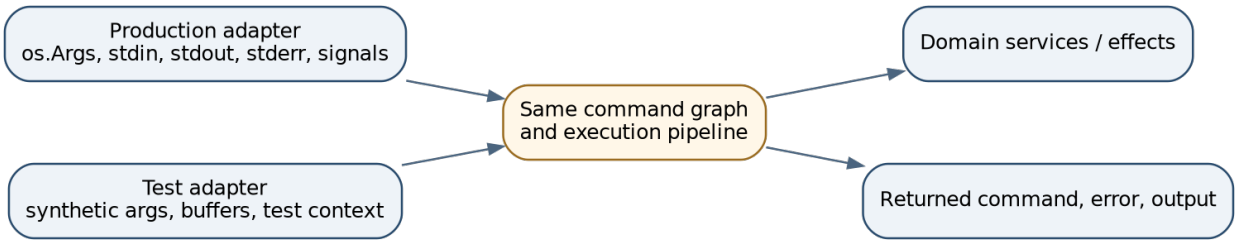

Figure 4.3: Production and test adapters inject different process boundaries into the same command graph and execution pipeline.

Definition 4.15 (Process boundary). A process boundary is an interface between the command executor and ambient operating-system facilities: argv, stdin, stdout, stderr, environment, signals, and exit status.

Definition 4.16 (Injectable boundary). A process boundary is injectable when a caller can provide an alternate implementation without modifying the executor's semantic logic.

Relevant API shapes include:

```go
func (c *Command) SetArgs(args []string)
func (c *Command) SetIn(r io.Reader)
func (c *Command) SetOut(w io.Writer)
func (c *Command) SetErr(w io.Writer)
func (c *Command) ExecuteContext(ctx context.Context) error
```

The OS values become fallbacks, not hard dependencies.

4.18 Substitution principle for tests

Let B_{prod} be production boundaries and B_{test} be synthetic boundaries. Let $E(T, B)$ denote execution of command graph T with boundaries B.

A useful testability property is:

$$\text{framework semantics of } E(T, B_{test}) \text{ are the same as } E(T, B_{prod}),$$

except for observations intentionally supplied by the boundary, such as token input and output destination.

> **Fundamentals - Observational equivalence.** Two executions are observationally equivalent with respect to an observer when that observer cannot distinguish them through the chosen outputs. Here the relevant observer sees selected commands, errors, phase traces, and emitted streams. We do not require equality of operating-system details that the test intentionally replaces.

This is not full observational equivalence because production domain services may differ. It means tests do not use a separate routing or validation engine.

A typical helper is:

```go
func execute(root *cobra.Command, args ...string) (string, error) {
    buf := new(bytes.Buffer)
    root.SetOut(buf)
    root.SetErr(buf)
    root.SetArgs(args)
    err := root.Execute()
    return buf.String(), err
}
```

This supports deterministic tests of command selection, aliases, validation, hook order, help, and context propagation.

4.19 Counterexample 3: split output channels

If framework code writes through `cmd.OutOrStdout()` but a handler calls `fmt.Println`, tests capture only part of the behavior. The program has two output authorities:

```
injected writer <- framework output
process stdout  <- handler output
```

The repair is consistency: application-visible output should use the injected boundary, while machine protocol output and diagnostics should use explicitly distinct injected channels.

4.20 Mutable models and test isolation

A command tree can retain state between executions:

- flags record whether they changed;
- defaults may have been synthesized;
- completion may have changed advisory flag metadata;
- commands may have been added or removed;
- caches may be populated.

Therefore factory-per-test construction is often safer than sharing one graph across a test table.

Definition 4.17 (Test isolation). A set of test executions is isolated when the initial semantic model and boundary state for one case do not depend on the order or result of earlier cases.

Reset methods can help, but constructing a fresh tree makes the initial condition explicit.

4.21 Documentation as an interpreter

Reference documentation is another projection of the command graph. Cobra's documentation package walks commands, renders usage, examples, local and inherited flags, and parent/child links, and emits Markdown, man pages, or reStructuredText.

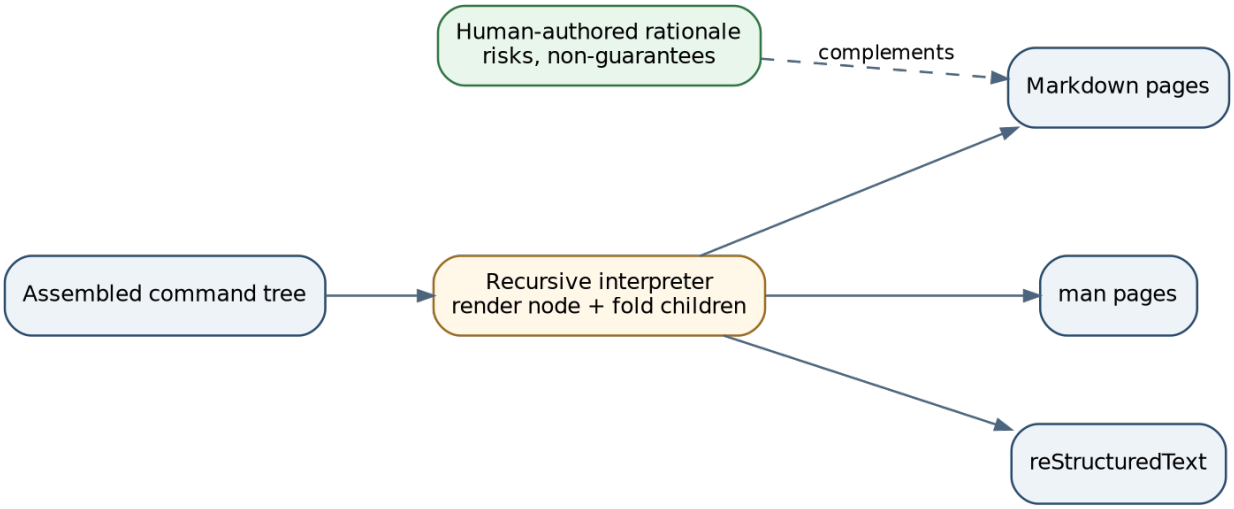

Figure 4.4: Documentation generation is a recursive interpreter over the assembled command tree; human-authored rationale complements rather than duplicates it.

A simplified API shape is:

```
func GenMarkdownTreeCustom(
    cmd *cobra.Command,
    dir string,
    filePrepender func(filename string) string,
    linkHandler func(link string) string,
) error
```

Definition 4.18 (Model-derived documentation). Documentation is model-derived when every structural reference fact it states - command path, usage, flags, inheritance, child links - is computed from the assembled runtime model rather than copied into an independently maintained catalog.

This creates a division of labor:

```
runtime model
    -> exact structural reference
human-authored text
    -> motivation, tradeoffs, failure modes, security assumptions
```

Generated documentation prevents one class of drift. It cannot infer whether an operation is dangerous, idempotent, transactional, or authorized unless those concepts are represented in metadata.

4.22 Documentation generation as a fold

For each node v, let $R(v)$ render its local page from metadata and effective policy views. A tree generator recursively computes:

$$\mathrm{Docs}(v) = \{R(v)\} \cup \bigcup_{c \in \mathrm{children}(v)} \mathrm{Docs}(c).$$

Link generation uses the same parent/child relation. The generator may filter nodes by availability.

The build must assemble the same model phase as production. If plugins or defaults are absent, the generated reference is structurally consistent with the wrong graph.

4.23 Artifact naming and injectivity

A generator maps command paths to filenames. Let

$$f : \mathrm{Paths}(T) \to \mathrm{Filenames}\,.$$

Definition 4.19 (Collision-free path encoding). Filename encoding f is collision-free when it is injective:

$$f(p_1) = f(p_2) \Rightarrow p_1 = p_2.$$

Flattening paths by replacing separators with hyphens can violate injectivity. For example, a nested path `sub third` and a single command named `sub-third` may map to the same filename under a naive encoding.

A collision-free scheme can:

- preserve directory hierarchy;
- length-prefix each component;
- escape separators and escape characters;
- append a stable path hash;
- verify uniqueness before writing.

This is a general generation law, not merely a documentation detail.

4.24 Worked example 4: one node, four observations

Take the `project add` node. The same model facts yield different observations:

Interpreter	Input	Output
dispatch	complete argv	selected node `add` and remaining args
help	node `add`	usage, description, local/inherited flags
completion	partial argv `project a`	candidate `add` plus directive
docs	assembled graph	`atlas_project_add.md` page and navigation links
tests	synthetic argv/context/streams	returned error, selected node, captured output

This table is the core architecture of the book. The interpreters differ because users ask different questions. They agree because command identity and scope come from one model.

4.25 A general architecture for interface frameworks

The Cobra study can be abstracted into five components.

Definition 4.20 (Executable interface architecture). An executable interface architecture is a tuple

$$\mathcal{A} = (M, R, S, X, \mathcal{J}),$$

where:

- M is the semantic model, such as a rooted command tree;
- R is name and path resolution;
- S is scoped policy resolution;
- X is operational execution semantics;
- $\mathcal{J}$ is a family of non-effect and effect interpreters, such as help, completion, validation, documentation, and execution.

The architecture should state four laws.

4.25.1 Shared-fact law

If interpreters need the same fact, it has one semantic authority in M.

4.25.2 Scope law

Effective policy follows an explicit lookup and shadowing rule.

4.25.3 Admission law

The primary effect begins only after named predicates and preparation stages succeed.

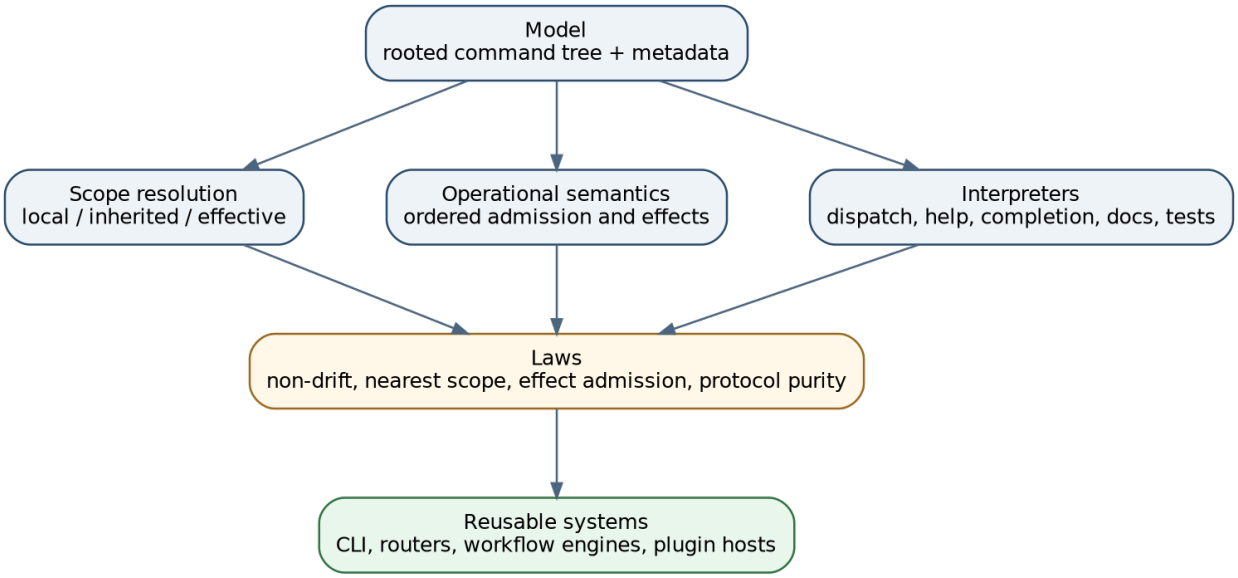

Figure 4.5: Synthesis of the four chapters: model, scope, operational semantics, and interpreters jointly establish reusable laws.

4.25.4 Projection law

Advisory and generated views are derived from M and do not replace authoritative runtime checks.

These laws apply to more than CLIs:

- HTTP routers with OpenAPI generation and request validation;
- RPC service registries with reflection and client stubs;
- workflow graphs with editors, validators, and executors;
- plugin catalogs with discovery, completion, and documentation;
- UI action trees with keyboard commands and help palettes;
- configuration schemas with forms and static analysis.

4.26 Capstone worked design: a minimal command framework

We now sketch a small language-neutral design that incorporates the four chapters.

4.26.1 Model

```
type Node = {
    name: String,
    aliases: Set<String>,
    children: List<Node>,
    localPolicy: Map<Key, Value>,
    metadata: Metadata,
    validator: Validator,
    handler: Handler?
}
```

4.26.2 Resolution

```
resolve(root, argv, matchPolicy) -> Result<SelectedNode, ResolutionError>
```

The result includes the selected node, canonical path, remaining tokens, and the spelling used by the caller.

4.26.3 Scope

```
effectivePolicy(node, key) -> Option<(value, provenanceNode)>
localPolicy(node) -> Map<Key, Value>
inheritedPolicy(node) -> Map<Key, Value>
```

4.26.4 Execution

```
execute(
    model: ResolvedModel,
    request: Invocation,
    boundaries: ProcessBoundaries,
    services: Services,
    policy: ExecutorPolicy
) -> ExecutionResult
```

`ExecutorPolicy` owns matching and hook traversal settings, avoiding package-global switches.

4.26.5 Constraints

```
type Constraint =
    | RequiredTogether(Set<FlagName>)
    | OneRequired(Set<FlagName>)
    | MutuallyExclusive(Set<FlagName>)
    | Custom(ConstraintId)

validate(constraints, assignment) -> List<Violation>
complete(constraints, partialAssignment) -> CandidatePolicy
```

Static constraints are shared. `Custom` constraints require an explicit runtime resolver and are labeled advisory when previewed.

4.26.6 Tooling query

```
completeQuery(model, partialArgv, context)
    -> { candidates, directive, diagnostics }
```

The response is serialized through a versioned, protocol-pure channel.

4.26.7 Documentation

```
generateReference(resolvedModel, pathEncoder, renderer)
    -> List<Artifact>
```

The generator first verifies path encoding injectivity.

4.26.8 Tests

```
newModelForTest() -> Model
execute(model, syntheticInvocation, bufferedBoundaries, fakeServices, policy)
```

Every test receives a fresh model and explicit dependencies.

This design is smaller than Cobra's compatibility surface but preserves the extracted laws.

4.27 Counterexample 4: generated reference without operational semantics

Suppose an OpenAPI or CLI generator accurately lists an endpoint or command but omits that:

- post-hooks do not run on handler error;
- completion providers may perform network queries;
- a destructive flag combination requires authorization;
- an operation is not idempotent.

The reference is structurally correct and operationally incomplete. Model-derived docs should be paired with authored explanations of non-guarantees, failure modes, and authority boundaries.

4.28 Counterexample 5: model-derived does not mean truthful

Descriptions and examples are themselves metadata written by humans. A generator can faithfully reproduce a false sentence. Generation ensures structural coupling, not semantic truth.

The corresponding quality strategy includes:

- review of descriptions and examples;
- tests for important usage strings;
- executable examples where feasible;
- design documentation for laws not encoded in the schema;
- explicit version and phase provenance for generated artifacts.

4.29 Design checklist

When adding an interpreter to a semantic model, ask:

- Is the interpreter authoritative or advisory?
- Which facts does it share with other interpreters?
- Are those facts represented once?
- Does it operate on complete or partial assignments?
- Does it mutate the model or return a derived view?
- What is the protocol grammar and which channel is machine-pure?
- Can the interpreter execute side effects, and should it?
- Are process boundaries injectable?
- Is test state isolated between executions?
- Does generated artifact naming preserve path identity?
- Which operational truths remain outside the model and require authored prose?

4.30 Chapter summary

A single semantic model can support several interpreters without flattening their authority. Constraint metadata becomes Boolean formulas evaluated over complete assignments by runtime

validation and over partial assignments by completion guidance. A hidden completion command turns the executable into a query server over a partial program, requiring a strict stdout grammar, explicit directives, bounded side effects, and runtime revalidation. Injectable process boundaries let tests run the production executor with synthetic inputs. Documentation generation is a recursive projection over the assembled graph and requires collision-free artifact naming. The general architecture combines model, scope, execution, and interpreter laws and applies to many interface frameworks beyond CLIs.

> **Source correspondence.** The concrete evidence for this chapter is concentrated in `flag_groups.go`, `flag_groups_test.go`, `completions.go`, `completions_test.go`, `command_test.go`, and `doc/md_docs.go`.

4.31 Exercises

4.31.1 Constraint logic

1. Write Boolean formulas for the following rules:
   - exactly two of four flags must be present;
   - `--template` implies `--region`;
   - `--dry-run` excludes `--commit` but neither is required;
   - if `--from` is set, exactly one of `--to` or `--duration` is required.
2. For each formula in Exercise 1, compute admissible next flags under at least two partial assignments.
3. Prove that combining one-required and mutually-exclusive on the same nonempty group expresses exactly-one.
4. Design a deterministic error-ordering policy for a set of violated constraints. Compare "declaration order," "lexicographic order," and "most specific first."

4.31.2 Completion protocols

5. Define a versioned completion response grammar that can carry candidates, descriptions, directives, and structured diagnostics without allowing newline injection.
6. A dynamic completion provider takes 800 ms and is called on every keystroke. Design a cancellation and caching strategy. State the cache key and invalidation rule.
7. Construct a protocol-pollution bug in which a debug log becomes a candidate. Write a test that detects any stdout record not accepted by the response grammar.
8. Explain why a partial command program cannot always be parsed by appending an arbitrary dummy token. Give a flag-value counterexample.

4.31.3 Testing and documentation

9. Design a property-based test that generates small command trees and checks that every visible command appears in documentation output exactly once.
10. Give an injective filename encoding for arbitrary path components containing spaces, hyphens, underscores, and percent signs. Prove or argue why it is injective.
11. A test suite reuses one command graph. One case sets `--verbose`, and a later case unexpectedly observes it as changed. Diagnose the state leak and propose two repairs.
12. Design a test boundary that captures stdout and stderr separately while also recording an ordered event trace. Why can two buffers alone be insufficient for ordering assertions?

4.31.4 Architecture synthesis

13. Choose one domain - HTTP routers, workflow engines, plugin systems, or UI action trees - and instantiate the tuple $\mathcal{A} = (M, R, S, X, \mathcal{J})$.

14. Identify one advisory interpreter and one authoritative interpreter in your chosen domain. State a bug caused by confusing them.
15. Design a phase model for plugins plus generated docs. Which phase is signed or deployed? How do you ensure production and documentation assemble the same plugin set?
16. **Capstone project.** Implement or specify a small command framework for the `atlas` language. It must support exact routing, aliases, local/persistent flags, nearest-scope lookup, composed positional validators, exact-one flag constraints, a completion query, injected streams/context, and generated Markdown reference pages. Include at least one counterexample test for each chapter's main law.

Glossary and Notation

This glossary collects terms after they have been motivated and applied in the chapters. It is a lookup aid, not a substitute for the worked explanations.

Term	Definition	First developed
Admission error	A rejection after parsing but before the main effect, such as invalid arguments, missing required flags, or failed preparation.	Chapter 3
Admission frontier	The transition after which an invocation is eligible to perform its primary domain effect.	Chapter 3
Admission predicate	The conjunction of conditions established before the primary effect begins.	Chapter 3
Advisory interpreter	An interpreter that guides a user or tool but does not authorize or admit the effect.	Chapter 4
Alias	An alternate token that matches a command node while preserving a canonical node identity.	Chapter 1
Authoritative interpreter	An interpreter whose decision controls the corresponding runtime effect or admission.	Chapter 4
Bit-set encoding	Representation of a finite set by assigning each atom a distinct bit and summing or OR-ing those bit values.	Chapter 4
Collision-free path encoding	An injective mapping from semantic paths to generated artifact names.	Chapter 4
Command path	The unique root-to-node sequence of command names in a rooted command tree.	Chapter 1
Command word	A finite sequence of command-name tokens intended to select a path in the command tree.	Chapter 1

Term	Definition	First developed
Command vocabulary	The finite set of command-name tokens used by the language.	Chapter 1
Complete assignment	A value assignment for every relevant flag variable.	Chapter 4
Completion directive	A compact protocol value telling an external shell how to treat completion candidates.	Chapter 4
Completion query	An interpreter from a partial command program and live model to candidates, directives, and diagnostics.	Chapter 4
Diagnostic determinism	Stable ordered error output for fixed model and input, independent of incidental iteration order.	Chapter 4
Effect error	An error returned after the main domain handler was entered.	Chapter 3
Effective environment	The result of folding local environments from leaf to root with local precedence and defaults last.	Chapter 2
Executable interface architecture	A model, resolver, scope system, operational semantics, and family of interpreters for one external interface.	Chapter 4
Guaranteed cleanup boundary	A scope in which registered cleanup runs on every covered return path.	Chapter 3
Inherited policy	A policy value declared on an ancestor and visible at a descendant because no nearer declaration shadows it.	Chapter 2
Injectable boundary	A process or runtime boundary that a caller can substitute without changing semantic execution logic.	Chapter 4
Interpreter	A function that reads the semantic model and produces one operational or descriptive view.	Chapter 1
Late synthesized default	A framework fallback added only at a phase where it is needed and only if the application has not claimed its slot.	Chapter 2

Term	Definition	First developed
Left-biased override	Partial-map combination in which a binding in the left map takes precedence over the right map.	Chapter 2
Legal extension	A complete assignment that agrees with a partial assignment and satisfies the constraint formula.	Chapter 4
Local environment	The finite partial map of policy declarations owned by one command node.	Chapter 2
Local flag	A flag owned by one command and not automatically inherited by descendants.	Chapter 2
Model-derived documentation	Reference documentation computed from the assembled runtime model rather than copied into a second schema.	Chapter 4
Match relation	The rule deciding whether a token denotes a command node through its primary name, aliases, or optional matching policies.	Chapter 1
Model phase	A named stage of construction or interpretation at which a particular view of a mutable model is resolved.	Chapter 1
Mutually-exclusive constraint	A Boolean relation requiring at most one member of a flag group to be present.	Chapter 4
Nearest-scope lookup	Resolution that selects a local declaration if present, otherwise the nearest ancestor declaration, otherwise a fallback.	Chapter 2
One-required constraint	A Boolean relation requiring at least one member of a flag group to be present.	Chapter 4
Parse error	Failure to assign valid flag and value structure to the tokens of a resolved command.	Chapter 3
Parsing authority	The component responsible for assigning syntactic roles and values to tokens.	Chapter 3
Partial assignment	An assignment in which some variables have values and others remain unassigned.	Chapter 4
Partial command program	An incomplete token sequence whose final command or flag value may be only a prefix.	Chapter 4

Term	Definition	First developed
Partial function	A function that may be undefined for some inputs; used to model failed command resolution.	Chapter 1
Persistent flag	A flag declared on a command and eligible to flow through its descendant subtree.	Chapter 2
Post-stage error	A failure in a lifecycle stage after the main effect has returned normally.	Chapter 3
Process boundary	The interface to argv, streams, context/signals, environment, and exit behavior.	Chapter 4
Process-global extension state	Behavior or registry state shared by every command tree in one process rather than scoped to one model or executor.	Chapter 2
Projection	A derived read-only view of a semantic model rather than a second editable authority.	Chapter 4
Protocol-pure stdout	Stdout containing only records accepted by the specified machine protocol.	Chapter 4
Provenance	The source node or fallback that supplied an effective policy value.	Chapter 2
Required-together constraint	A Boolean relation requiring either none or all members of a flag group to be present.	Chapter 4
Resolution error	Failure or ambiguity while mapping command tokens to a node.	Chapter 3
Scope of a declaration	The subtree of nodes at which a declaration may be visible before shadowing and declaration-kind restrictions.	Chapter 2
Semantic authority	The representation from which all subsystems derive a shared fact.	Chapter 1
Semantic slot	A framework-recognized role, such as the help flag or help command, that may be claimed by an application or filled by a fallback.	Chapter 2
Semantic command graph	A rooted labeled command topology plus metadata interpreted by routing, help, validation, completion, and docs.	Chapter 1
Shadowing	A local declaration taking precedence over an ancestor declaration for the same key.	Chapter 2

Term	Definition	First developed
Small-step transition	One legal move between execution configurations in an operational semantics.	Chapter 3
Test isolation	Independence of one test's initial model and boundary state from earlier test executions.	Chapter 4
Trace determinism	Stable framework phase and diagnostic sequence for fixed deterministic inputs and dependencies.	Chapter 3
Tree fold	A structurally recursive interpreter that combines a node's metadata with results from its children.	Chapter 1
Validator conjunction	Ordered short-circuit composition of validation predicates with first-error diagnostics.	Chapter 3

Symbols

Symbol	Meaning
$T = (V, E, r, \lambda, \mu)$	semantic command graph
V	command nodes
E	parent-to-child edges
r	root command
$\lambda(v)$	primary command name of node v
$\mu(v)$	metadata attached to node v
L_v	local policy environment at node v
Γ_v	effective policy environment at node v
$A \rhd B$	left-biased override; A wins
κ	invocation configuration
$\kappa \to \kappa'$	one small-step execution transition
α	complete flag assignment
ρ	partial flag assignment
$\alpha \vDash \varphi$	assignment α satisfies formula φ
$\mathrm{Ext}(\rho, \varphi)$	legal complete extensions of partial assignment ρ

Selected Hints and Solutions

The following are not complete solutions to every exercise. They model the level of reasoning expected and provide checkpoints for self-study.

Chapter 1

Exercise 2: tree or DAG

Attaching one node object beneath two parents violates the unique-parent property. Consequently, `Parent()`, inherited policy, and canonical `CommandPath()` become ambiguous. One repair is to create two distinct wrapper nodes that delegate to one shared handler. Another is to admit a DAG but define identity as a path occurrence rather than node object; this requires every interpreter to carry the occurrence path explicitly.

Exercise 6: ambiguity proof

A prefix p uniquely selects name n_i exactly when p is a prefix of n_i and not a prefix of any other name. A trie computes shortest unique prefixes efficiently. Building the trie is $O(L)$ in total name length; marking subtree leaf counts lets each name's shortest unique prefix be found in time proportional to its length.

Exercise 7: anti-drift property

Let R be the set of canonical paths accepted by dispatch under a bounded generated test vocabulary, and D the set of paths represented by generated documentation artifacts. A useful property is visible$(R) = D$. The test should derive both sets independently from interpreter outputs while using the same assembled graph.

Chapter 2

Exercise 1: associativity

Fix arbitrary key k. If $A(k)$ is defined, both parenthesizations return $A(k)$. Otherwise, if $B(k)$ is defined, both return $B(k)$. Otherwise both return $C(k)$ if defined and undefined otherwise. Since the maps agree at every key, they are equal.

Exercise 4: idempotent defaults

Sufficient conditions for $\mathcal{D}_p$ include: (1) synthesis checks whether each semantic slot is already claimed; (2) every inserted default marks or occupies that slot in the same way the check recognizes; (3) synthesis does not remove or rename claims. After one application, every applicable slot is claimed, so the second application performs no changes.

Exercise 11: global switch isolation

Move matching policy into an executor value:

```
type MatchPolicy = { prefix: Bool, caseInsensitive: Bool }
execute(model, invocation, boundaries, services, matchPolicy)
```

Resolution receives the policy explicitly. Two executors can share a process while selecting different relations without racing on package state.

Chapter 3

Exercise 1: failure rules

Argument failure transitions from `ArgCheck` directly to `Failed` and records an admission error; no pre-hook or effect has run. `RunE` failure transitions from `Effect` to `Failed` and records an effect error; earlier preparation and possibly external effects have occurred. The distinction is observable and should appear in the error type.

Exercise 4: diagnostic non-commutativity

Let P require exactly one argument and Q require every argument to be a known project name. On empty input, P reports an arity error while Q may vacuously succeed. On two unknown inputs, the order chooses between arity and membership diagnostics even though $P \wedge Q$ is logically the same formula.

Exercise 11: irreversible pre-hook

If a pre-hook sends a notification and a later flag-group check fails, the user observes a rejected command that still caused a notification. Repairs include moving the static flag check before pre-hooks, moving notification into the effect stage, or making the pre-hook prepare a message without sending it.

Chapter 4

Exercise 3: exact-one proof

One-required gives $\sum b_i \geq 1$. Mutually-exclusive gives $\sum b_i \leq 1$. Since the sum is an integer, both hold exactly when $\sum b_i = 1$.

Exercise 7: protocol pollution test

Execute the completion query with synthetic boundaries. Parse stdout according to the response grammar and assert complete consumption: every line except the last must be a valid candidate record, and the last must be a directive record. Any unconsumed line, including debug text, fails the test.

Exercise 10: injective encoding

One solution encodes each path component as `<byte-length>:<percent-encoded-bytes>` and joins components with `/`. Length prefixes make component boundaries unique; percent encoding makes the textual alphabet safe. Decoding is deterministic, so equal encodings imply equal component sequences.

Source Map and Further Reading

Primary Cobra source areas

The architectural claims in this book are grounded in the following files at commit `adbc8813901bba65827259daa`

- `command.go`: command topology, routing, execution pipeline, inheritance, streams, help, defaults, flags, and lifecycle.
- `args.go`: positional validator functions and conjunction.
- `flag_groups.go`: annotation-backed group constraints, deterministic validation, and completion interpretation.
- `completions.go`: hidden completion protocol, partial parsing, dynamic providers, and directive bit map.
- `doc/md_docs.go`: recursive Markdown generation from the runtime model.
- `command_test.go`: routing, context, aliases, hooks, streams, and execution behavior.
- `flag_groups_test.go`: group validation across local, persistent, and inherited flags.
- `completions_test.go`: completion compatibility behavior.

Architecture Garden source study

The evidence-backed project study and eight focused pattern notes are in the Cobra directory of the `go-go-parc` Software Architecture Garden at commit `15cda404dfdf4ca86ce69cd348a0388c0b9b5e73`:

- `Cobra architecture study`
- `Command Graph as Semantic Authority`
- `Hierarchical Policy Inheritance with Local Shadowing`
- `Staged Command Execution Pipeline`
- `Late Defaults with User Override`
- `Constraint Metadata Drives Validation and Completion`
- `Hidden Protocol Commands for Interactive Tooling`
- `Injectable Process Boundaries for Deterministic Tests`
- `Generate Documentation from the Runtime Model`

Suggested theoretical reading

The book's mathematical tools are standard and can be deepened through texts on:

- data structures and graph algorithms for rooted trees, tries, and traversal;
- programming-language semantics for environments, substitution, and small-step transition systems;
- logic and constraint satisfaction for Boolean formulas and partial assignments;
- functional programming for folds over recursive data;
- software architecture for dependency injection, protocol design, and generated interfaces;

- testing theory for observational equivalence, property-based tests, and deterministic traces.

The important methodological habit is to move between three levels without confusing them:

1. the concrete API and runtime behavior;
2. the abstract structure and law it implements;
3. the evidence that the implementation actually satisfies the law, including counterexamples and non-guarantees.